TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ

BÁO CÁO BÀI TẬP LỚN
KĨ THUẬT LẬP TRÌNH NÂNG CAO

NGÀNH : KỸ THUẬT MÁY TÍNH

HỆ : ĐẠI HỌC CHÍNH QUY

ĐỀ TÀI: MÔ PHỎNG THUẬT TOÁN A* TRONG BÀI TOÁN TÌM ĐƯỜNG TRÊN LƯỚI (ĐỀ TÀI 48)

THÁI NGUYÊN - 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BÁO CÁO BÀI TẬP LỚN

KĨ THUẬT LẬP TRÌNH NÂNG CAO

NĂM HỌC: 2025-2026

BỘ MÔN: Công nghệ thông tin

ĐỀ TÀI: MÔ PHỎNG THUẬT TOÁN A* TRONG BÀI TOÁN TÌM ĐƯỜNG TRÊN LƯỚI (ĐỀ TÀI 48)

GIÁO VIÊN HƯỚNG DẪN : GV. Nghiêm Văn Tính

HỌ VÀ TÊN SINH VIÊN : Trần Văn Khải

LỚP : K59.KTP

MSSV : K235480106035

THÁI NGUYÊN - 2026

KHOA ĐIỆN TỬ
BỘ MÔN: CÔNG NGHỆ THÔNG TIN

NHIỆM VỤ BÀI TẬP LỚN

Sinh viên: Trần Văn Khải Lớp: K59.KTP MSSV: K235480106035

Giáo viên hướng dẫn: GV. Nghiêm Văn Tính

1. Tên đề tài

MÔ PHỎNG THUẬT TOÁN A* TRONG BÀI TOÁN TÌM ĐƯỜNG TRÊN LƯỚI (ĐỀ TÀI 48)

2. Các số liệu ban đầu

Bộ dữ liệu lưới mẫu gồm sáu bản đồ có kích thước và mật độ vật cản khác nhau, trong đó có bản đồ địa hình mang trọng số chi phí di chuyển.

3. Nội dung các phần thuyết minh và tính toán

- Phân tích bài toán tìm đường trên lưới, phát biểu mô hình đồ thị và hàm chi phí.
- Trình bày cơ sở lý thuyết của thuật toán A*, các hàm ước lượng h(n) và tính khả dụng.
- Chứng minh tính tối ưu của A* và phân tích độ phức tạp thời gian, bộ nhớ.
- Thiết kế chương trình gồm kiến trúc phân tầng, sơ đồ lớp và cấu trúc dữ liệu.
- Cài đặt chương trình có giao diện, kiểm thử và thực nghiệm đo đạc trên dữ liệu mẫu.

4. Các sản phẩm, kết quả

5. Ngày giao nhiệm vụ: 20/09/2026

6. Ngày hoàn thành nhiệm vụ: 27/10/2026

BỘ MÔN (Ký, ghi rõ họ tên) **GIÁO VIÊN HƯỚNG DẪN (Ký, ghi rõ họ tên)**

LỜI NÓI ĐẦU

Bài toán tìm đường trên lưới là một trong những bài toán kinh điển của khoa học máy tính, xuất hiện ở hầu hết các hệ thống có yếu tố di chuyển trong không gian rời rạc như trò chơi điện tử, robot tự hành, phần mềm chỉ đường, mô phỏng giao thông hay điều phối xe tự động trong kho hàng. Đặc điểm chung của nhóm bài toán này là không gian tìm kiếm lớn, yêu cầu thời gian phản hồi ngắn và đòi hỏi đường đi tìm được phải đúng, hợp lệ và tốt nhất theo một tiêu chí chi phí nào đó.

Thuật toán A* do Hart, Nilsson và Raphael công bố năm 1968 là lời giải được sử dụng rộng rãi nhất cho nhóm bài toán nêu trên. Điểm cốt lõi của A* là việc đánh giá mỗi đỉnh bằng hàm ưu tiên f(n) = g(n) + h(n), trong đó g(n) là chi phí thực đã bỏ ra để đi từ đỉnh xuất phát tới n, còn h(n) là ước lượng chi phí từ n tới đích. Nhờ thành phần h(n), A* tập trung tìm kiếm về phía có khả năng chứa lời giải thay vì lan ra mọi hướng như tìm kiếm theo chiều rộng, và vẫn giữ được tính tối ưu khi h(n) thỏa mãn điều kiện khả dụng.

Trong khuôn khổ học phần Kĩ thuật lập trình nâng cao, em chọn Đề tài 48 với nội dung mô phỏng thuật toán A* trong bài toán tìm đường trên lưới. Mục tiêu của đề tài không dừng ở việc cài đặt một hàm tìm đường cho ra kết quả đúng, mà còn phải làm rõ cơ sở lý thuyết của thuật toán, phân tích độ phức tạp, so sánh với các thuật toán đối chứng, và quan trọng nhất là xây dựng một chương trình hoàn chỉnh có giao diện cho phép quan sát trực quan từng bước mà thuật toán thực hiện.

Chương trình được cài đặt bằng ngôn ngữ Python, chỉ sử dụng thư viện chuẩn, trong đó giao diện đồ hoạ dùng Tkinter. Bộ máy tìm kiếm được thiết kế theo hướng phát sự kiện, cho phép giao diện hiển thị tập đỉnh đang chờ, tập đỉnh đã duyệt và đường đi cuối cùng theo thời gian thực. Toàn bộ số liệu dùng cho phần thực nghiệm của báo cáo do chính chương trình sinh ra và được lưu lại thành tệp CSV, JSON, HTML để có thể kiểm chứng lại.

Nội dung báo cáo gồm năm chương. Chương 1 trình bày bối cảnh, phát biểu bài toán, khảo sát các giải pháp hiện hành và xác định mục tiêu, phạm vi của đề tài. Chương 2 trình bày cơ sở lý thuyết về không gian trạng thái, các hàm ước lượng h(n), thuật toán A*, chứng minh tính tối ưu và phân tích độ phức tạp. Chương 3 trình bày phân tích thiết kế chương trình gồm kiến trúc phân tầng, sơ đồ lớp, cấu trúc dữ liệu lưới và thiết kế giao diện. Chương 4 trình bày việc cài đặt, các kịch bản

kiểm thử và năm nhóm thí nghiệm đo đạc kèm phân tích kết quả. Chương 5 tổng kết kết quả đạt được, nêu hạn chế và đề xuất hướng phát triển.

Trong quá trình thực hiện, em đã cố gắng vận dụng kiến thức của học phần vào từng bước của đề tài, từ việc lựa chọn cấu trúc dữ liệu, phân tích độ phức tạp, đến việc kiểm chứng kết quả bằng thực nghiệm. Do kiến thức và thời gian còn hạn chế, báo cáo khó tránh khỏi thiếu sót, em rất mong nhận được ý kiến góp ý của thầy để hoàn thiện hơn.

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Bảng dưới đây liệt kê các thuật ngữ và từ viết tắt được sử dụng thống nhất trong toàn bộ báo cáo. Các thuật ngữ tiếng Anh chuyên ngành được giữ nguyên ở dạng gốc và giải thích nghĩa tiếng Việt ở lần xuất hiện đầu tiên.

Thuật ngữ	**Giải thích**
A*	Thuật toán tìm kiếm có thông tin dùng hàm ưu tiên f(n) = g(n) + h(n)
BFS	Breadth First Search, tìm kiếm theo chiều rộng, mọi bước cùng chi phí 1
UCS	Uniform Cost Search, tìm kiếm theo chi phí đều, tương đương Dijkstra
GBFS	Greedy Best First Search, tìm kiếm tham lam chỉ dựa vào h(n)
OPEN	Tập đỉnh đã sinh nhưng chưa duyệt, còn gọi là biên tìm kiếm (frontier)
CLOSED	Tập đỉnh đã được lấy ra và xét xong
g(n)	Chi phí thực tế nhỏ nhất tạm thời từ đỉnh xuất phát tới n
h(n)	Hàm ước lượng chi phí từ n tới đích, còn gọi là heuristic
f(n)	Giá trị ưu tiên của đỉnh n trong hàng đợi, f(n) = g(n) + w.h(n)
w	Trọng số nhân vào h(n) trong biến thể weighted A*, mặc định bằng 1
Khả dụng	Tính chất h(n) không ước lượng vượt quá chi phí thực tới đích
Nhất quán	Tính chất h(n) thỏa bất đẳng thức tam giác giữa hai đỉnh kề
Lưới	Đồ thị rời rạc hai chiều, mỗi ô là một đỉnh, có 4 hoặc 8 ô kề
Ô kề	Ô lân cận theo 4 hướng hoặc 8 hướng của một ô
Cắt góc	Hành vi cho phép đi chéo xuyên qua khe hở giữa hai ô tường
GUI	Graphical User Interface, giao diện đồ hoạ người dùng
CSV	Comma Separated Values, định dạng tệp văn bản phân tách bằng dấu
BTL	Bài tập lớn
TNUT	Trường Đại học Kỹ thuật Công nghiệp Thái Nguyên

Bảng 0.1: Danh mục thuật ngữ và từ viết tắt dùng trong báo cáo

Ngoài các thuật ngữ trên, báo cáo dùng thống nhất đơn vị chi phí không có thứ nguyên. Một bước đi sang ô kề theo bốn hướng có chi phí bằng trọng số của ô được đi vào. Một bước đi chéo có chi phí bằng căn bậc hai của hai nhân với trọng số của ô được đi vào, phản ánh đúng tỉ lệ khoảng cách hình học giữa cạnh và đường chéo của hình vuông đơn vị.

MỤC LỤC

LỜI NÓI ĐẦU....1
DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT....3
MỤC LỤC....4
DANH MỤC BẢNG BIỂU....4
DANH MỤC HÌNH ẢNH....4
CHƯƠNG 1: TỔNG QUAN VÀ PHÁT BIỂU BÀI TOÁN....4
1.1. Bối cảnh của bài toán tìm đường trên lưới....4
1.2. Phát biểu bài toán và các ký hiệu....4
1.3. Khảo sát các giải pháp hiện hành....9
1.4. Các thách thức kĩ thuật của đề tài....11
1.5. Mục tiêu, đối tượng và phạm vi của đề tài....11
1.6. Bố cục của báo cáo....12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ THUẬT TOÁN A*....13
2.1. Không gian trạng thái và mô hình đồ thị của lưới....13
2.2. Hàm ưu tiên và họ thuật toán tìm kiếm....13
2.3. Hàm ước lượng h(n) và các tính chất....15
2.4. Thuật toán A* chi tiết và chứng minh tính tối ưu....17
2.5. Phân tích độ phức tạp của thuật toán....20
2.6. Nhận xét về phạm vi hiệu quả của A*....21
2.7. Các biến thể và hướng mở rộng liên quan....22
CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ CHƯƠNG TRÌNH....23
3.1. Yêu cầu chức năng và yêu cầu phi chức năng....23
3.2. Kiến trúc phân tầng của chương trình....23
3.3. Thiết kế lớp và các mô-đun chính....24
3.4. Cấu trúc dữ liệu biểu diễn lưới....26
3.5. Cấu trúc dữ liệu hàng đợi ưu tiên và các bảng tra cứu....27
3.6. Thiết kế giao diện người dùng....27
3.7. Bộ dữ liệu mẫu và kịch bản thực nghiệm....29
CHƯƠNG 4: CÀI ĐẶT, THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ....30
4.1. Môi trường cài đặt và công cụ sử dụng....30
4.2. Sáu bộ dữ liệu mẫu....30
4.3. Diễn biến quá trình tìm kiếm trên lưới vật cản....32
4.4. Giao diện chương trình....33
4.5. Kịch bản kiểm thử và kết quả kiểm thử....37
4.6. Thí nghiệm 1 và 2: so sánh A* với các thuật toán đối chứng....39
4.7. Thí nghiệm 3: ảnh hưởng của hàm ước lượng....42
4.8. Thí nghiệm 4: biến thể A* có trọng số....43

4.9. Thí nghiệm 5: ảnh hưởng của cấu hình bốn hướng và tám hướng.....45
4.10. Phân tích tổng hợp và độ tin cậy của số liệu.....47
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....48
5.1. Kết quả đã đạt được.....48
5.2. Hạn chế của đề tài.....48
5.3. Hướng phát triển.....49
TÀI LIỆU THAM KHẢO.....50
PHỤ LỤC A: MÃ NGUỒN CỐT LÕI.....51
A.1. Hàm tìm kiếm của bộ máy A*, UCS, Greedy và BFS.....51
A.2. Phần cập nhật trạng thái mô phỏng của giao diện.....53
A.3. Cách chạy chương trình.....54

DANH MỤC BẢNG BIỂU

Bảng 0.1: Danh mục thuật ngữ và từ viết tắt dùng trong báo cáo.....3
Bảng 1.1: Các ký hiệu dùng trong báo cáo.....6
Bảng 1.2: So sánh ưu, nhược điểm của các nhóm giải pháp tìm đường trên lưới.....10
Bảng 2.1: Các hàm ước lượng được cài đặt và tính chất.....16
Bảng 2.2: Tổng hợp độ phức tạp của các thuật toán tìm kiếm trên lưới.....21
Bảng 3.1: Đặc tả các hàm chính của tầng xử lý.....26
Bảng 3.2: Quy ước ký tự trong tệp biểu diễn lưới.....27
Bảng 4.1: Môi trường cài đặt và cấu hình thử nghiệm.....30
Bảng 4.2: Thông số của sáu bộ dữ liệu mẫu.....32
Bảng 4.4: Kịch bản kiểm thử tiêu biểu.....38
Bảng 4.5: Tổng hợp kết quả kiểm thử theo nhóm.....39
Bảng 4.6: Kết quả thí nghiệm 1 và 2 trên sáu bộ dữ liệu mẫu.....40
Bảng 4.7: Mức giảm số đỉnh duyệt của A* so với tìm kiếm theo chi phí đều.....42
Bảng 4.8: Số đỉnh A* phải duyệt theo từng hàm ước lượng.....43
Bảng 4.9: Kết quả khảo sát biến thể A* có trọng số.....44
Bảng 4.10: Kết quả so sánh cấu hình bốn hướng và tám hướng.....46

DANH MỤC HÌNH ẢNH

Hình 2.1: Vòng đời trạng thái của một đỉnh trong quá trình tìm kiếm, từ chưa biết sang tập mở, sang tập đóng và kết thúc khi gặp đích.....15
Hình 2.2: Lưu đồ thuật toán A* từ khâu khởi tạo, vòng lặp lấy đỉnh ưu tiên, cập nhật đỉnh kề cho tới khi trả về đường đi hoặc kết luận vô nghiệm.....17
Hình 2.3: Minh hoạ các đại lượng g(n), h(n), f(n) tại một bước của A* trên lưới có vật cản….19
Hình 3.1: Kiến trúc phân tầng của chương trình mô phỏng thuật toán A*.....24
Hình 3.2: Sơ đồ lớp lõi của thư viện tìm kiếm và quan hệ giữa các thành phần.....25
Hình 3.3: Trình tự mô phỏng từng bước giữa giao diện và bộ máy tìm kiếm.....28
Hình 4.1: Sáu bộ dữ liệu mẫu dùng cho thực nghiệm, kèm kích thước và tỉ lệ tường.....31
Hình 4.2: Diễn biến quá trình A* trên lưới vật cản ngẫu nhiên qua bốn thời điểm.....33
Hình 4.3: Giao diện chương trình sau khi hoàn tất tìm kiếm trên bộ dữ liệu mê cung.....34
Hình 4.4: Trạng thái giữa chừng của quá trình tìm kiếm với hai tập OPEN và CLOSED.....35
Hình 4.5: Hộp thoại so sánh bốn thuật toán trên cùng một bản đồ.....35
Hình 4.6: Bộ dữ liệu địa hình có trọng số với cấu hình tám hướng.....36
Hình 4.7: Thẻ hướng dẫn và chú thích màu trong giao diện chương trình.....37
Hình 4.8: Số đỉnh đã duyệt của bốn thuật toán trên sáu bộ dữ liệu mẫu.....41
Hình 4.9: Số đỉnh A* phải duyệt theo từng hàm ước lượng, lưới 4 hướng.....43
Hình 4.10: Đánh đổi giữa số đỉnh đã duyệt và chi phí đường đi theo trọng số w.....45
Hình 4.11: Ảnh hưởng của việc cho phép đi chéo tới chi phí và số bước đi.....46

CHƯƠNG 1: TỔNG QUAN VÀ PHÁT BIỂU BÀI TOÁN

1.1. Bối cảnh của bài toán tìm đường trên lưới

Tìm đường là bài toán xuất hiện mỗi khi một tác nhân cần di chuyển từ một vị trí ban đầu tới một vị trí đích trong một không gian có chướng ngại. Trong thực tế, không gian đó có thể là mặt bằng của một nhà kho, mạng lưới đường phố của một thành phố, bản đồ của một màn chơi điện tử, hay vùng làm việc của một cánh tay robot. Điểm chung của các không gian này là chúng đều được rời rạc hoá thành các đơn vị nhỏ, và việc rời rạc hoá như vậy dẫn trực tiếp tới mô hình lưới hai chiều với mỗi ô là một trạng thái.

Mô hình lưới có nhiều ưu điểm về mặt kĩ thuật. Thứ nhất, việc xác định các ô kề của một ô chỉ cần cộng trừ toạ độ theo một bảng hướng cố định, nhờ đó không cần lưu trữ danh sách cạnh tường minh như với đồ thị tổng quát. Thứ hai, lưới cho phép gán thêm trọng số cho từng ô để mô tả địa hình có chi phí di chuyển khác nhau như đường nhựa, bãi cỏ, cát lún hay bùn lầy. Thứ ba, kết quả tìm kiếm trên lưới rất dễ trực quan hoá, điều này đặc biệt phù hợp với mục tiêu mô phỏng của đề tài.

Trong bối cảnh học tập, yêu cầu tìm đường trên lưới còn là ví dụ gọn gàng để minh hoạ mối quan hệ giữa ba nhóm thuật toán duyệt đồ thị cơ bản. Tìm kiếm theo chiều rộng tối ưu khi mọi bước đi có cùng chi phí. Tìm kiếm theo chi phí đều tối ưu khi các bước đi có chi phí khác nhau nhưng phải duyệt theo mọi hướng. A* giữ được tính tối ưu của tìm kiếm theo chi phí đều trong khi định hướng tìm kiếm về phía đích nhờ hàm ước lượng. Việc đặt ba thuật toán cạnh nhau trên cùng một bộ dữ liệu cho phép định lượng rõ ràng lợi ích mà tri thức đặc thù của bài toán mang lại.

1.2. Phát biểu bài toán và các ký hiệu

Bài toán được phát biểu như sau. Cho một lưới chữ nhật gồm H hàng và W cột. Mỗi ô được xác định bằng cặp toạ độ (r, c) với r là chỉ số hàng và c là chỉ số cột. Mỗi ô có một trọng số w(r, c) là chi phí để đi vào ô đó. Ô bị chặn, còn gọi là tường, được biểu diễn bằng trọng số vô cùng. Hai ô được gọi là kề nhau nếu toạ độ của chúng khác nhau không quá một đơn vị trên mỗi trục, trong đó cấu hình bốn hướng chỉ xét các ô kề cạnh còn cấu hình tám hướng xét thêm các ô kề góc.

Cho một ô xuất phát S và một ô đích G, yêu cầu của bài toán là tìm một dãy ô liên tiếp S = n0, n1, ..., nk = G sao cho mỗi cặp ô liên tiếp đều kề nhau, không ô nào

là tường, và tổng chi phí của dãy là nhỏ nhất có thể. Chi phí của một bước đi từ ô a sang ô b được tính bằng hệ số khoảng cách của bước đi nhân với trọng số của ô đích b. Hệ số khoảng cách bằng một khi bước đi theo bốn hướng và bằng căn bậc hai của hai khi bước đi chéo. Trường hợp không tồn tại dãy nào thoả mãn các điều kiện trên, chương trình phải kết luận rõ ràng rằng không có đường đi.

Bảng 1.1 tổng hợp các ký hiệu được dùng xuyên suốt báo cáo. Việc cố định ký hiệu ngay từ đầu giúp các công thức ở Chương 2 và các bảng số liệu ở Chương 4 luôn được hiểu theo cùng một nghĩa.

Ký hiệu	**Ý nghĩa**	**Mặc định**
H, W	Số hàng và số cột của lưới	25 và 40
(r, c)	Toạ độ của một ô, r là hàng, c là cột	gốc ở góc trên trái
S, G	Ô xuất phát và ô đích	góc dưới trái và góc trên phải
w(r, c)	Trọng số của ô, chi phí để đi vào ô đó	1,0; ô tường bằng vô cùng
c(a, b)	Chi phí của một bước đi từ ô a sang ô b	1,0 hoặc căn hai của hai
g(n)	Chi phí thực nhỏ nhất tạm thời từ S tới n	khởi tạo bằng vô cùng
h(n)	Hàm ước lượng chi phí từ n tới G	chọn trong bảng 2.1
f(n)	Giá trị ưu tiên dùng xếp hàng đợi	g(n) cộng w nhân h(n)
b	Hệ số phân nhánh, số ô kề trung bình	4 hoặc 8 theo cấu hình
d	Độ sâu của lời giải, số bước trên đường đi tối ưu	tuỳ bộ dữ liệu

Bảng 1.1: Các ký hiệu dùng trong báo cáo

Với cách phát biểu trên, lưới được xem như một đồ thị có hướng với tập đỉnh là các ô đi được và tập cạnh là các cặp ô kề hợp lệ. Trọng số của cạnh phụ thuộc vào hướng đi và trọng số của ô đích, do đó đồ thị có trọng số không âm. Tính chất không âm này là điều kiện để các thuật toán tìm kiếm theo chi phí và A* cho kết quả tối ưu.

1.3. Khảo sát các giải pháp hiện hành

Trước khi cài đặt, em đã khảo sát bốn nhóm giải pháp thường được dùng cho bài toán tìm đường trên lưới. Nhóm thứ nhất là tìm kiếm không có thông tin gồm tìm kiếm theo chiều rộng và tìm kiếm theo chi phí đều hay còn gọi là Dijkstra. Hai thuật toán này đảm bảo tìm được đường đi tốt nhất theo tiêu chí chi phí tương ứng nhưng không sử dụng bất kì hiểu biết nào về vị trí của đích, vì vậy chúng lan ra theo

mọi hướng và duyệt số đỉnh xấp xỉ diện tích của vùng liên thông chứa đỉnh xuất phát.

Nhóm thứ hai là tìm kiếm tham lam, đại diện là Greedy Best First Search. Thuật toán này chỉ dùng h(n) để quyết định đỉnh nào được xét tiếp, nhờ đó tốc độ rất nhanh trên nhiều bản đồ nhưng không có bảo đảm về chất lượng đường đi. Trên các bản đồ có vật cản dạng phòng kín hoặc hành lang quanh co, thuật toán này dễ bị hút vào một hướng cụt và buộc phải quay lại với chi phí lớn hơn mức tối ưu.

Nhóm thứ ba là tìm kiếm có thông tin, đại diện là A*. Thuật toán kết hợp chi phí đã bỏ ra với ước lượng chi phí còn lại, nhờ đó vừa giữ được tính tối ưu khi hàm ước lượng khả dụng, vừa giảm mạnh số đỉnh phải duyệt so với tìm kiếm theo chi phí đều. Biến thể weighted A* nhân thêm trọng số lớn hơn một vào h(n) để đổi chất lượng đường đi lấy tốc độ, phù hợp với các ứng dụng cần phản hồi nhanh.

Nhóm thứ tư là các kĩ thuật tối ưu chuyên biệt cho lưới như tìm kiếm theo bước nhảy Jump Point Search, tìm kiếm A* phân cấp cho bản đồ lớn, và các thuật toán tìm kiếm gia tăng như D* Lite dùng khi bản đồ thay đổi trong lúc di chuyển. Các kĩ thuật này cho hiệu năng cao hơn nhưng phụ thuộc vào giả thiết rất cụ thể về dữ liệu, chẳng hạn lưới phải đồng nhất về chi phí hoặc phải có cấu trúc phân cấp sẵn. Bảng 1.2 tóm tắt ưu điểm và hạn chế của từng nhóm.

Nhóm giải pháp	**Ưu điểm**	**Hạn chế**
Tìm kiếm theo chiều rộng	Đơn giản, đúng đắn, tối ưu theo số bước	Chỉ đúng khi mọi bước cùng chi phí, duyệt rất nhiều đỉnh
Tìm kiếm theo chi phí đều (Dijkstra)	Tối ưu với trọng số không âm, ổn định	Không định hướng, duyệt gần như toàn bộ vùng liên thông
Greedy Best First	Nhanh, ít bộ nhớ, dễ cài đặt	Không bảo đảm tối ưu, chất lượng đường đi dao động mạnh
A* và weighted A*	Tối ưu khi h khả dụng, giảm mạnh số đỉnh duyệt	Chất lượng phụ thuộc h, bộ nhớ tăng theo số đỉnh sinh
Jump Point Search, A* phân cấp	Rất nhanh trên lưới lớn, đồng nhất	Giả thiết chặt về dữ liệu, cài đặt phức tạp
D* Lite và A* gia tăng	Thích nghi khi bản đồ thay đổi	Phức tạp, chỉ cần thiết cho bài toán động

Bảng 1.2: So sánh ưu, nhược điểm của các nhóm giải pháp tìm đường trên lưới

1.4. Các thách thức kĩ thuật của đề tài

Thách thức thứ nhất là việc lựa chọn hàm ước lượng. Một hàm ước lượng quá thấp sẽ khiến A* suy giảm thành tìm kiếm theo chi phí đều và mất đi lợi thế định hướng. Một hàm ước lượng quá cao có thể vượt quá chi phí thực tế, làm mất tính tối ưu và cho ra đường đi không còn là tốt nhất. Vì vậy đề tài phải cài đặt nhiều hàm ước lượng khác nhau, có cơ chế đánh giá tính khả dụng theo cấu hình lưới, và có thực nghiệm đo đạc để thấy rõ ảnh hưởng của từng hàm.

Thách thức thứ hai là ảnh hưởng của cấu hình di chuyển. Khi cho phép đi chéo, chi phí của một bước chéo phải lớn hơn chi phí của một bước thẳng để phản ánh đúng khoảng cách hình học. Nếu đặt sai hệ số này, đường đi tìm được sẽ có hình dạng sai lệch rõ rệt. Ngoài ra cần xử lý trường hợp đi chéo xuyên qua khe hở giữa hai ô tường, một tình huống có thể khiến đường đi lọt qua vách mỏng và không còn hợp lệ về mặt hình học.

Thách thức thứ ba là yêu cầu trực quan hoá. Muốn quan sát được thuật toán đang làm gì, chương trình phải cho phép tạm dừng giữa chừng, hiển thị đồng thời tập đỉnh đang chờ, tập đỉnh đã duyệt và đỉnh đang xét, đồng thời cập nhật các đại lượng g(n), h(n), f(n) theo thời gian thực. Điều này đòi hỏi bộ máy tìm kiếm phải được thiết kế theo hướng phát sự kiện thay vì chạy một mạch từ đầu đến cuối.

Thách thức thứ tư là tính trung thực và khả năng kiểm chứng của số liệu. Các kết luận trong báo cáo phải dựa trên số liệu đo được từ chính chương trình, không được suy đoán. Vì vậy toàn bộ thí nghiệm được thực hiện bằng một kịch bản tự động chạy trên sáu bộ dữ liệu mẫu, kết quả được ghi ra tệp CSV và tệp báo cáo HTML để có thể chạy lại và đối chiếu bất cứ lúc nào.

1.5. Mục tiêu, đối tượng và phạm vi của đề tài

Mục tiêu của đề tài gồm bốn điểm. Điểm thứ nhất là trình bày đầy đủ cơ sở lý thuyết của thuật toán A* gồm định nghĩa hàm ưu tiên, điều kiện khả dụng và nhất quán của hàm ước lượng, chứng minh tính tối ưu và phân tích độ phức tạp thời gian cùng bộ nhớ. Điểm thứ hai là xây dựng chương trình hoàn chỉnh bằng Python có giao diện đồ hoạ cho phép chỉnh sửa lưới, chọn thuật toán và quan sát quá trình tìm kiếm theo từng bước.

Điểm thứ ba là chuẩn bị một bộ dữ liệu mẫu đa dạng, bao phủ các tình huống khác nhau như lưới trống, vật cản rải rác, mê cung, cấu trúc phòng, hành lang xoắn ốc và địa hình có trọng số, nhằm bộc lộ điểm mạnh và điểm yếu của từng thuật toán. Điểm thứ tư là thực hiện các thí nghiệm so sánh có định lượng, bao gồm so sánh A* với các thuật toán đối chứng, so sánh các hàm ước lượng, khảo sát biến thể weighted A* và khảo sát ảnh hưởng của việc cho phép đi chéo.

Đối tượng nghiên cứu của đề tài là các thuật toán tìm kiếm trên đồ thị có trọng số không âm, cụ thể là A*, tìm kiếm theo chi phí đều, tìm kiếm tham lam và tìm kiếm theo chiều rộng. Phạm vi nghiên cứu giới hạn ở không gian hai chiều dạng lưới với kích thước tối đa khoảng sáu mươi ô mỗi chiều, số bước đi giới hạn theo bốn hoặc tám hướng, chi phí địa hình nhận một trong các giá trị một, hai, bốn hoặc sáu. Đề tài không xét các biến thể tìm kiếm trong không gian liên tục, không xét tìm đường cho nhiều tác nhân cùng lúc và không xét bản đồ thay đổi theo thời gian.

1.6. Bố cục của báo cáo

Báo cáo được tổ chức thành năm chương. Chương 1 giới thiệu bối cảnh, phát biểu bài toán, khảo sát các giải pháp hiện hành và xác định mục tiêu, phạm vi nghiên cứu. Chương 2 trình bày cơ sở lý thuyết, trong đó trọng tâm là các hàm ước lượng, thuật toán A* và phân tích độ phức tạp. Chương 3 trình bày phân tích thiết kế chương trình gồm kiến trúc phân tầng, sơ đồ lớp, cấu trúc dữ liệu và thiết kế giao diện.

Chương 4 trình bày kết quả cài đặt và thực nghiệm. Chương này mô tả môi trường cài đặt, sáu bộ dữ liệu mẫu, giao diện chương trình, bộ kịch bản kiểm thử và năm nhóm thí nghiệm đo đạc. Kết quả của mỗi nhóm thí nghiệm được trình bày dưới dạng bảng số liệu kèm biểu đồ và phần phân tích nguyên nhân. Chương 5 tổng kết những kết quả đã đạt được, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo.

Phần cuối báo cáo gồm danh mục tài liệu tham khảo và phụ lục mã nguồn. Phụ lục A trích dẫn phần mã nguồn cốt lõi của thuật toán A* và của giao diện để người đọc có thể đối chiếu giữa phần trình bày lý thuyết và phần cài đặt thực tế.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ THUẬT TOÁN A*

2.1. Không gian trạng thái và mô hình đồ thị của lưới

Một bài toán tìm kiếm được mô hình hoá bằng bộ bốn thành phần gồm trạng thái ban đầu, tập hành động, hàm chuyển trạng thái và hàm chi phí. Với bài toán tìm đường trên lưới, trạng thái ban đầu là ô xuất phát S, tập trạng thái là tất cả các ô đi được, tập hành động của một ô là các bước di chuyển sang ô kề, hàm chuyển trạng thái trả về ô kề tương ứng, và hàm chi phí trả về chi phí của bước đi như đã định nghĩa ở mục 1.2.

Bộ bốn thành phần này tương đương với một đồ thị có hướng G = (V, E) trong đó mỗi đỉnh của V là một ô đi được và mỗi cạnh của E là một cặp ô kề hợp lệ. Với cấu hình bốn hướng, mỗi đỉnh trong vùng bên trong lưới có đúng bốn đỉnh kề, do đó hệ số phân nhánh trung bình của đồ thị xấp xỉ bốn. Với cấu hình tám hướng, hệ số phân nhánh xấp xỉ tám nhưng số cạnh bị chặn trên bởi ba phần tư số đỉnh khi áp dụng quy tắc không cắt góc. Trên lưới có vật cản, hệ số phân nhánh thực tế nhỏ hơn giá trị lý thuyết và phụ thuộc vào mật độ tường của bộ dữ liệu.

Độ sâu của lời giải, ký hiệu là d, bằng số bước đi trên đường đi tối ưu. Với lưới kích thước H hàng và W cột, độ sâu tối đa trong trường hợp xấu nhất có thể lên tới H nhân W khi đường đi buộc phải đi qua gần hết các ô, điển hình là bộ dữ liệu hành lang xoắn ốc được trình bày ở Chương 4. Các đại lượng b và d này là hai tham số quyết định độ phức tạp của mọi thuật toán tìm kiếm được phân tích trong chương này.

Một đặc điểm quan trọng của lưới là cấu trúc lân cận rất đều đặn. Vì toạ độ của ô kề luôn bằng toạ độ của ô hiện tại cộng thêm một vectơ trong bảng hướng cố định, việc sinh đỉnh kề chỉ tốn thời gian hằng số cho mỗi đỉnh. Trong cài đặt của đề tài, danh sách đỉnh kề của một ô còn được lưu đệm theo ô, nhờ đó các lần sinh lân cận lặp lại không phải tính lại chi phí. Đây là một tối ưu quan trọng vì trong tìm kiếm có mở lại đỉnh, cùng một ô có thể được sinh kề nhiều lần.

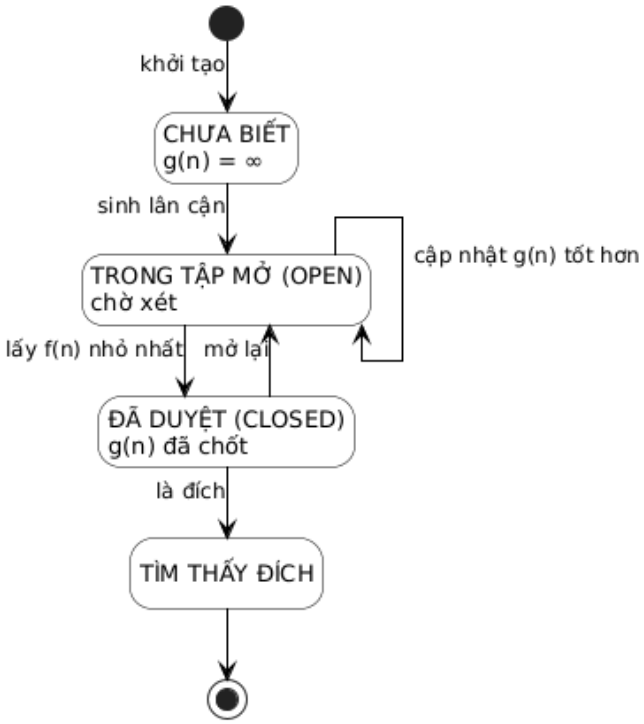
2.2. Hàm ưu tiên và họ thuật toán tìm kiếm

Hầu hết thuật toán tìm kiếm trên đồ thị có trọng số đều được mô tả bằng một khung chung. Khung này duy trì hai tập đỉnh, tập OPEN chứa các đỉnh đã được sinh ra và đang chờ xét, và tập CLOSED chứa các đỉnh đã được lấy ra xử lý. Ở mỗi vòng

lặp, thuật toán lấy ra khỏi tập OPEN một đỉnh theo một quy tắc xác định bởi hàm ưu tiên, đưa đỉnh đó vào tập CLOSED, kiểm tra điều kiện dừng, rồi sinh các đỉnh kề và cập nhật chi phí của chúng. Sự khác biệt giữa các thuật toán nằm ở hàm ưu tiên và ở chính sách cập nhật đỉnh đã có trong tập OPEN hoặc tập CLOSED.

Tìm kiếm theo chiều rộng coi mọi bước đi có cùng chi phí bằng một và dùng hàng đợi hai đầu theo nguyên tắc vào trước ra trước. Tìm kiếm theo chi phí đều dùng hàng đợi ưu tiên với khoá là g(n), nhờ đó đỉnh có chi phí đã bỏ ra nhỏ nhất luôn được xét trước. Tìm kiếm tham lam dùng khoá là h(n), chỉ quan tâm tới ước lượng khoảng cách còn lại. A* dùng khoá là tổng của hai thành phần, tức là kết hợp thông tin thực tế với thông tin ước lượng.

Trạng thái của một đỉnh trong quá trình tìm kiếm được mô tả bằng sáu bước chuyển như Hình 2.1. Ban đầu mọi đỉnh nằm ở trạng thái chưa biết với giá trị g bằng vô cùng. Khi một đỉnh kề được sinh ra và có chi phí tốt hơn giá trị hiện có, đỉnh đó chuyển sang trạng thái nằm trong tập OPEN. Đỉnh ở trong tập OPEN có thể được cập nhật nhiều lần nếu tìm thấy đường đi rẻ hơn. Khi đỉnh được lấy ra theo tiêu chí ưu tiên, đỉnh chuyển sang tập CLOSED và giá trị g của nó được coi là đã chốt. Nếu hàm ước lượng không nhất quán, một đỉnh đã ở tập CLOSED vẫn có thể được mở lại khi phát hiện đường đi tốt hơn. Quá trình kết thúc khi đỉnh được xét chính là đích, hoặc khi tập OPEN rỗng tức là không tồn tại đường đi.



Hình 2.1: Vòng đời trạng thái của một đỉnh trong quá trình tìm kiếm, từ chưa biết sang tập mở, sang tập đóng và kết thúc khi gặp đích

Việc phân biệt rõ tập OPEN và tập CLOSED có ý nghĩa cả về lý thuyết lẫn về mặt cài đặt. Về lý thuyết, tính đúng đắn của A* dựa trên việc mỗi đỉnh chỉ được chốt giá trị g khi nó thực sự có chi phí nhỏ nhất có thể. Về mặt cài đặt, hai tập này còn là cơ sở để giao diện tô màu, giúp người học nhìn thấy trực tiếp mức độ lan rộng của từng thuật toán trên cùng một bản đồ.

2.3. Hàm ước lượng h(n) và các tính chất

Hàm ước lượng h(n) là thành phần mang thông tin đặc thù của bài toán vào thuật toán tìm kiếm. Để A* giữ được tính tối ưu, h(n) phải thoả mãn điều kiện khả dụng, nghĩa là với mọi đỉnh n, giá trị h(n) không được lớn hơn chi phí thực nhỏ nhất từ n tới đích. Trên lưới, điều kiện này được đảm bảo bằng cách nhân khoảng cách hình học với chi phí bước nhỏ nhất của lưới. Nếu lưới có địa hình trọng số lớn hơn một, việc nhân với chi phí bước nhỏ nhất vẫn giữ được tính khả dụng vì mọi bước đi thực tế đều có chi phí không nhỏ hơn giá trị đó.

Một tính chất mạnh hơn của hàm ước lượng là tính nhất quán, còn gọi là điều kiện đơn điệu. Hàm h được gọi là nhất quán nếu với mọi cặp đỉnh kề nhau a và b, giá trị h(a) không lớn hơn tổng của chi phí bước đi từ a tới b và giá trị h(b). Điều

kiện này chính là dạng rời rạc của bất đẳng thức tam giác. Khi h nhất quán, giá trị f(n) dọc theo bất kì đường đi nào cũng không giảm, do đó A* chỉ cần duyệt mỗi đỉnh một lần và không phải mở lại đỉnh đã đóng. Tất cả các hàm ước lượng dùng cho lưới bốn hướng và tám hướng trong đề tài đều nhất quán khi hệ số chi phí bước đi được đặt đúng.

$$h(n) \leq h^*(n) \tag{2.1}$$

$$h(a) \leq c(a, b) + h(b) \tag{2.2}$$

Bốn hàm ước lượng được cài đặt trong đề tài gồm khoảng cách Manhattan, khoảng cách Euclid, khoảng cách Chebyshev và khoảng cách octile. Khoảng cách Manhattan là tổng của hiệu số hàng và hiệu số cột, phù hợp với lưới bốn hướng và cho giá trị chính xác khi không có vật cản. Khoảng cách Euclid là khoảng cách theo đường thẳng, luôn nhỏ hơn hoặc bằng chi phí thực nên khả dụng cho cả hai cấu hình nhưng thường lỏng hơn khoảng cách Manhattan trên lưới bốn hướng.

Khoảng cách Chebyshev là giá trị lớn nhất của hiệu số hàng và hiệu số cột, phù hợp với lưới tám hướng khi bước chéo có chi phí bằng một. Khoảng cách octile là trường hợp đặc biệt quan trọng vì nó khớp chính xác với chi phí thực của lưới tám hướng, trong đó mỗi bước chéo có chi phí bằng căn bậc hai của hai. Vì khớp chính xác nên khoảng cách octile là hàm ước lượng chặt nhất mà vẫn khả dụng cho lưới tám hướng, và trong thực nghiệm ở Chương 4 nó cho số đỉnh duyệt ít nhất. Bảng 2.1 tóm tắt tính chất của từng hàm.

Hàm h(n)	**Công thức**	**Khả dụng cho**
Manhattan	h = |dr| + |dc|	Lưới 4 hướng, bước thẳng chi phí 1
Euclid	h = căn của (dr^2 + dc^2)	Cả lưới 4 hướng và 8 hướng
Chebyshev	h = max(|dr|, |dc|)	Lưới 8 hướng khi bước chéo chi phí 1
Octile	h = (dr + dc) + (căn hai trừ hai) nhân min(dr, dc)	Lưới 8 hướng, bước chéo chi phí căn hai
Trung tính	h = 0	Mọi lưới, khi đó A* suy giảm thành Dijkstra

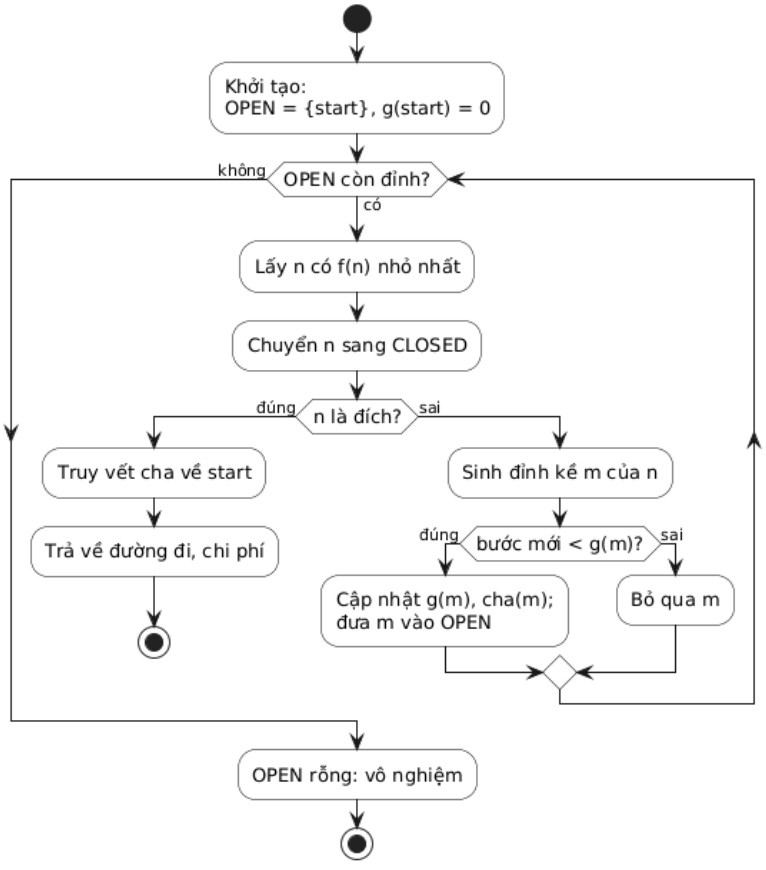
Bảng 2.1: Các hàm ước lượng được cài đặt và tính chất

Cần lưu ý rằng khoảng cách Manhattan không khả dụng trên lưới tám hướng. Lý do là trên lưới tám hướng, chi phí thực để đi từ một ô tới một ô chéo kề chỉ là

căn bậc hai của hai, trong khi khoảng cách Manhattan lại báo giá trị bằng hai. Giá trị ước lượng vượt quá chi phí thực khiến A* có thể chọn nhầm đỉnh và cho ra đường đi không tối ưu. Chương trình vì vậy có cơ chế cảnh báo trên giao diện mỗi khi người dùng chọn tổ hợp thuật toán và hàm ước lượng không đảm bảo tính khả dụng.

2.4. Thuật toán A* chi tiết và chứng minh tính tối ưu

A* là trường hợp riêng của khung tìm kiếm nêu ở mục 2.2 với hàm ưu tiên là tổng của chi phí đã bỏ ra và chi phí ước lượng còn lại. Quy trình của thuật toán gồm việc khởi tạo tập OPEN với đỉnh xuất phát, lặp lại việc lấy ra đỉnh có giá trị ưu tiên nhỏ nhất, kiểm tra đích, sinh đỉnh kề và cập nhật chi phí. Hình 2.2 trình bày lưu đồ của thuật toán với đầy đủ các nhánh xử lý.



Hình 2.2: Lưu đồ thuật toán A từ khâu khởi tạo, vòng lặp lấy đỉnh ưu tiên, cập nhật đỉnh kề cho tới khi trả về đường đi hoặc kết luận vô nghiệm*

Cấu trúc dữ liệu cho tập OPEN là hàng đợi ưu tiên cài bằng cây vun đống nhị phân, cho phép lấy ra đỉnh có khoá nhỏ nhất và thêm đỉnh mới với độ phức tạp

logarit. Song song với hàng đợi, thuật toán lưu một bảng giá trị g cho các đỉnh đã sinh và một bảng cha để truy vết đường đi khi kết thúc. Việc dùng bảng băm cho hai bảng này giúp thao tác tra cứu và cập nhật có độ phức tạp trung bình hằng số.

Một chi tiết kĩ thuật ảnh hưởng rõ rệt tới thời gian chạy của A* là cách phá vỡ thế bằng của các giá trị ưu tiên. Trên lưới trống, rất nhiều đỉnh có cùng giá trị f, và thứ tự chúng được lấy ra có thể làm số đỉnh duyệt thay đổi đáng kể dù chi phí đường đi cuối cùng không đổi. Trong cài đặt của đề tài, khi hai đỉnh có cùng f thì đỉnh có h nhỏ hơn được ưu tiên trước, tức là ưu tiên đỉnh gần đích hơn. Kỹ thuật này giữ nguyên tính tối ưu nhưng thường rút ngắn đáng kể số đỉnh phải duyệt.

Một chi tiết kĩ thuật khác là chính sách mở lại đỉnh. Khi hàm ước lượng kém chặt, chẳng hạn trong weighted A* với trọng số lớn hơn một, giá trị ưu tiên có thể không đơn điệu và một đỉnh đã ở tập CLOSED vẫn có thể được tìm thấy với chi phí g nhỏ hơn. Chương trình cho phép bật hoặc tắt chính sách này. Khi bật, thuật toán trung thực về mặt lý thuyết nhưng có thể phải xét lại nhiều đỉnh. Khi tắt, thuật toán chạy nhanh hơn nhưng chất lượng đường đi có thể giảm. Ảnh hưởng của hai chế độ này được định lượng ở mục 4.8.

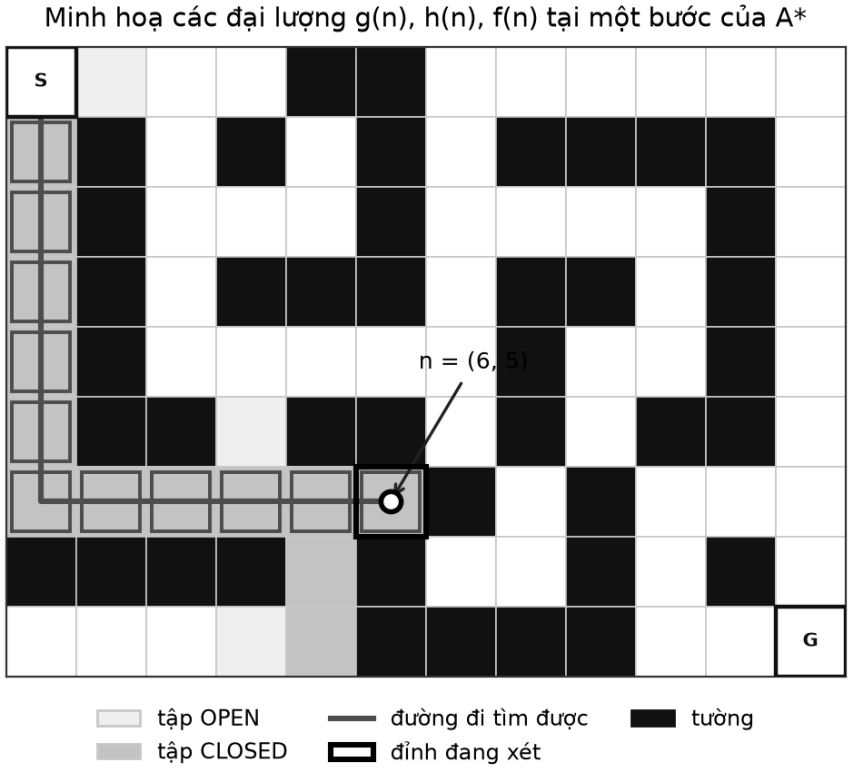
Trước khi trình bày chứng minh tính tối ưu, cần nêu một bổ đề cơ bản. Với hàm ước lượng khả dụng, mọi đỉnh n được lấy ra khỏi tập OPEN để đưa vào tập CLOSED đều có giá trị g(n) đúng bằng chi phí nhỏ nhất từ đỉnh xuất phát tới n. Chứng minh bổ đề này được thực hiện bằng phản chứng. Giả sử tồn tại đỉnh đầu tiên khi được lấy ra có giá trị g lớn hơn chi phí thực tối ưu. Gọi đường đi tối ưu tới đỉnh đó có đỉnh cuối cùng nằm trong tập OPEN là m. Đỉnh m có giá trị g đúng bằng chi phí tối ưu tới nó theo giả thiết về tính đầu tiên. Khi đó giá trị ưu tiên của m bằng chi phí tối ưu tới m cộng với ước lượng từ m tới đích, không vượt quá chi phí tối ưu của toàn đường đi, trong khi giá trị ưu tiên của đỉnh được lấy ra lại lớn hơn. Vì hàng đợi ưu tiên luôn lấy đỉnh có khoá nhỏ nhất, m phải được lấy ra trước, mâu thuẫn với giả thiết. Bổ đề được chứng minh.

Tính tối ưu của A* là hệ quả trực tiếp của bổ đề trên. Giả sử đỉnh đích được lấy ra khỏi tập OPEN khi vẫn còn một đường đi rẻ hơn chưa được khám phá. Theo bổ đề, giá trị g của đích lúc đó đúng bằng chi phí nhỏ nhất tìm được tới đích, mà giá trị này lại lớn hơn chi phí của đường đi rẻ hơn. Lập luận tương tự chứng minh bổ đề cho thấy phải tồn tại một đỉnh trên đường đi rẻ hơn có giá trị ưu tiên nhỏ hơn, và

đỉnh đó sẽ được lấy ra trước đích, dẫn tới mâu thuẫn. Vì vậy A* với hàm ước lượng khả dụng luôn trả về đường đi có chi phí nhỏ nhất. Kết quả này cũng được kiểm chứng bằng thực nghiệm ở mục 4.6, nơi chi phí của A* luôn trùng khớp với chi phí của tìm kiếm theo chi phí đều trên cả sáu bộ dữ liệu.

$$f(n) = g(n) + w \cdot h(n) \qquad (2.3)$$

Trong công thức trên, w là trọng số của biến thể A* có trọng số. Khi w bằng một thì hàm ưu tiên trở về dạng cổ điển f(n) = g(n) + h(n) và thuật toán giữ nguyên tính tối ưu nếu hàm ước lượng khả dụng. Khi w lớn hơn một thì thuật toán ưu tiên đi nhanh về phía đích, đổi lại đường đi có thể không còn là đường rẻ nhất. Ảnh hưởng định lượng của trọng số w được khảo sát ở mục 4.8.



Hình 2.3: Minh hoạ các đại lượng g(n), h(n), f(n) tại một bước của A trên lưới có vật cản*

Hình 2.3 minh hoạ cách các đại lượng xuất hiện trong một bước chạy thực tế. Tại bước thứ mười bốn của quá trình tìm kiếm, đỉnh đang xét là ô có toạ độ hàng sáu cột năm, với chi phí đã bỏ ra bằng mười một, ước lượng khoảng cách tới đích bằng tám và giá trị ưu tiên bằng mười chín. Vùng màu xám đậm là các đỉnh đã

duyệt xong, vùng xám nhạt là các đỉnh đang chờ trong tập OPEN, và đường kẻ đậm là đường đi tạm thời tốt nhất tính tới thời điểm đó. Hình này cũng cho thấy A* không lan đều ra mọi hướng mà tập trung phát triển về phía đích.

2.5. Phân tích độ phức tạp của thuật toán

Độ phức tạp thời gian của A* trong trường hợp xấu nhất là O(b mũ d), trong đó b là hệ số phân nhánh và d là độ sâu của lời giải. Đây là chặn trên tương ứng với trường hợp hàm ước lượng không mang lại thông tin, tức là h(n) bằng không trên toàn bộ lưới. Khi đó A* suy giảm thành tìm kiếm theo chi phí đều và phải duyệt toàn bộ vùng liên thông. Ngược lại, khi hàm ước lượng khớp chính xác với chi phí thực, nghĩa là h(n) bằng h*(n) với mọi đỉnh, A* chỉ duyệt đúng các đỉnh nằm trên đường đi tối ưu và độ phức tạp giảm xuống còn O(d).

Trường hợp thực tế nằm giữa hai cực trị trên. Nếu hàm ước lượng khả dụng và sai số của nó bị chặn bởi một hằng số, số đỉnh duyệt có thể được chứng minh là bậc đa thức theo độ sâu. Trên lưới, khi hàm ước lượng khả dụng và khá chặt, vùng đỉnh được duyệt có dạng gần giống một hình bầu dục kéo dài giữa đỉnh xuất phát và đỉnh đích, do đó số đỉnh duyệt xấp xỉ tỉ lệ với số ô nằm trong hình bầu dục đó chứ không phải toàn bộ lưới. Mức độ thu hẹp này phụ thuộc vào chiều dài đường đi và mật độ vật cản.

Độ phức tạp bộ nhớ của A* là O(b mũ d) do thuật toán phải lưu toàn bộ tập OPEN và tập CLOSED. Đây là hạn chế lớn nhất của A* trong các bài toán có không gian trạng thái khổng lồ, và cũng là động lực ra đời của các biến thể tiết kiệm bộ nhớ như tìm kiếm sâu dần theo ngưỡng chi phí và A* có giới hạn bộ nhớ. Với đề tài này, lưới có kích thước tối đa khoảng một nghìn tám trăm ô nên bộ nhớ không phải là yếu tố giới hạn. Tuy nhiên việc nêu rõ đặc điểm này là cần thiết để giải thích vì sao trong thực nghiệm, biểu đồ kích thước tập OPEN lớn nhất lại là một trong các chỉ số được ghi nhận.

Chi phí của từng thao tác cơ bản cũng cần được xem xét. Thao tác lấy đỉnh có khoá nhỏ nhất khỏi cây vun đống tốn thời gian logarit theo kích thước tập OPEN. Thao tác sinh đỉnh kề tốn thời gian hằng số cho mỗi đỉnh kề nhờ bảng hướng cố định và bộ đệm lân cận. Thao tác tra cứu và cập nhật bảng giá trị g có độ phức tạp trung bình hằng số. Vì vậy độ phức tạp của một vòng lặp chính là logarit theo số đỉnh sinh ra, và tổng thời gian chạy tỉ lệ với số đỉnh sinh nhân với logarit của số

đỉnh sinh. Kết quả đo đạc ở Chương 4 phù hợp với phân tích này khi thời gian chạy bám sát số đỉnh đã duyệt.

Thuật toán	Hàm ưu tiên	Thời gian xấu nhất	Bộ nhớ	Tối ưu
Tìm kiếm theo chiều rộng	vào trước ra trước, chi phí bước bằng 1	O(b mũ d)	O(b mũ d)	Chỉ khi mọi bước cùng chi phí
Tìm kiếm theo chi phí đều	g(n)	O(b mũ d)	O(b mũ d)	Có, với trọng số không âm
Greedy Best First	h(n)	O(b mũ d)	O(b mũ d)	Không
A*	g(n) cộng h(n)	O(b mũ d), tốt hơn nhiều trên thực tế	O(b mũ d)	Có, khi h khả dụng
A* có trọng số w	g(n) cộng w nhân h(n)	O(b mũ d)	O(b mũ d)	Không, chi phí tối đa bằng w lần tối ưu

Bảng 2.2: Tổng hợp độ phức tạp của các thuật toán tìm kiếm trên lưới

2.6. Nhận xét về phạm vi hiệu quả của A*

Từ phân tích ở mục 2.5 có thể rút ra ba nhận xét định hướng cho phần thực nghiệm. Nhận xét thứ nhất là lợi ích của A* so với tìm kiếm theo chi phí đều tỉ lệ với độ chặt của hàm ước lượng. Trên lưới trống với cấu hình bốn hướng, khoảng cách Manhattan là hàm ước lượng khớp chính xác nên số đỉnh duyệt giảm tới hơn mười lần. Trên cùng bộ dữ liệu đó, khoảng cách Euclid lỏng hơn đáng kể nên số đỉnh duyệt gần bằng trường hợp không dùng thông tin. Đây là lý do vì sao việc chọn hàm ước lượng phù hợp với cấu hình di chuyển quan trọng không kém việc cài đặt đúng thuật toán.

Nhận xét thứ hai là cấu trúc của bản đồ có thể vô hiệu hoá lợi ích của hàm ước lượng. Trên bộ dữ liệu hành lang xoắn ốc, toàn bộ lưới chỉ có một lối đi duy nhất từ đỉnh xuất phát tới đích, do đó mọi thuật toán buộc phải duyệt gần hết số ô. Trong trường hợp này, hàm ước lượng dù chặt tới đâu cũng không giúp giảm số đỉnh duyệt, điều này cho thấy chất lượng của thuật toán tìm kiếm luôn phụ thuộc vào cấu trúc của bài toán chứ không chỉ phụ thuộc vào bản thân thuật toán.

Nhận xét thứ ba là việc cho phép đi chéo làm giảm rõ rệt chi phí đường đi nhưng làm tăng số đỉnh cần duyệt trên mỗi bước, bởi hệ số phân nhánh tăng từ bốn lên tám. Hai xu hướng này ngược chiều nhau, và lựa chọn cấu hình nào phụ thuộc vào yêu cầu của ứng dụng. Với ứng dụng mô phỏng chuyển động của robot trong mặt bằng, cấu hình tám hướng thường cho đường đi tự nhiên hơn vì cho phép di chuyển theo đường chéo. Với ứng dụng mô phỏng di chuyển trên hệ thống đường phố, cấu hình bốn hướng lại phù hợp hơn vì các tuyến đường thực tế vuông góc với nhau.

2.7. Các biến thể và hướng mở rộng liên quan

A* có nhiều biến thể được sử dụng rộng rãi trong thực tế. Biến thể A* có trọng số nhân thêm hệ số lớn hơn một vào hàm ước lượng, nhờ đó giảm số đỉnh duyệt nhưng phải đánh đổi bằng chất lượng đường đi. Trên các lưới có địa hình trọng số, hành vi của biến thể này phức tạp hơn so với lưới đồng nhất và sẽ được phân tích kĩ ở mục 4.8. Biến thể A* với chính sách không mở lại đỉnh đã đóng thường được dùng trong công nghiệp vì đơn giản và ổn định, dù về mặt lý thuyết nó có thể cho kết quả không tối ưu trong một số trường hợp.

Các biến thể phức tạp hơn gồm tìm kiếm theo bước nhảy Jump Point Search, hoạt động bằng cách nhảy qua các ô có cùng hướng đi trên lưới đồng nhất để giảm số đỉnh sinh, tìm kiếm A* phân cấp cho bản đồ lớn bằng cách chia bản đồ thành nhiều tầng, và A* gia tăng như D* Lite cho bản đồ thay đổi theo thời gian. Các biến thể này đều là hướng phát triển tiềm năng của đề tài và được đề cập ở mục 5.3.

Về phương diện học tập, việc cài đặt và so sánh bốn thuật toán trên cùng một khung tìm kiếm giúp làm rõ rằng tìm kiếm theo chiều rộng, tìm kiếm theo chi phí đều và A* chỉ khác nhau ở hàm ưu tiên và ở chính sách cập nhật đỉnh, trong khi toàn bộ khung duyệt đồ thị là giống nhau. Cách nhìn thống nhất này cũng là cơ sở để thiết kế mô-đun tìm kiếm của chương trình, được trình bày ở Chương 3.

CHƯƠNG 3: PHÂN TÍCH THIẾT KẾ CHƯƠNG TRÌNH

3.1. Yêu cầu chức năng và yêu cầu phi chức năng

Từ mục tiêu đã xác định ở Chương 1, chương trình được thiết kế để đáp ứng một bộ yêu cầu chức năng rõ ràng. Về mặt mô phỏng, chương trình phải cài đặt được thuật toán A* cùng ba thuật toán đối chứng gồm tìm kiếm theo chi phí đều, tìm kiếm tham lam và tìm kiếm theo chiều rộng. Về mặt cấu hình, người dùng phải chọn được hàm ước lượng, chọn giữa lưới bốn hướng và tám hướng, và điều chỉnh được trọng số của biến thể A* có trọng số.

Về mặt dữ liệu, chương trình phải cho phép chỉnh sửa lưới trực tiếp bằng chuột, sinh lưới mới theo mật độ vật cản, sinh mê cung ngẫu nhiên, nạp sáu bộ dữ liệu mẫu dựng sẵn, đọc và ghi lưới dưới dạng tệp văn bản. Về mặt quan sát, chương trình phải hiển thị theo thời gian thực tập đỉnh đang chờ, tập đỉnh đã duyệt, đỉnh đang xét và đường đi cuối cùng, kèm bảng số liệu gồm các giá trị g, h, f của đỉnh đang xét.

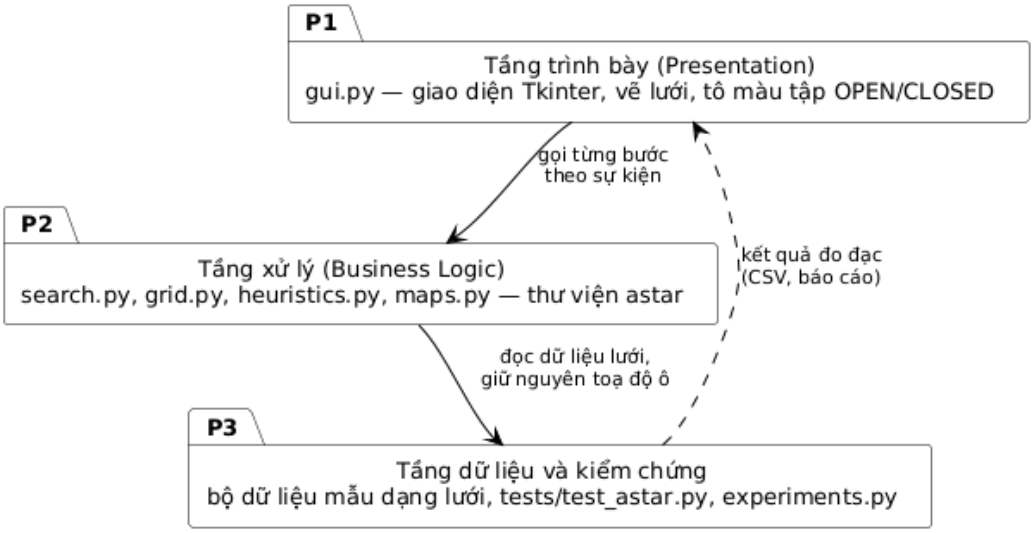
Về mặt đo đạc, chương trình phải xuất được bảng so sánh bốn thuật toán trên bản đồ hiện hành và ghi kết quả ra tệp CSV để phục vụ việc viết báo cáo. Ngoài giao diện, đề tài còn có một kịch bản dòng lệnh chạy toàn bộ thực nghiệm trên sáu bộ dữ liệu mẫu và sinh ra báo cáo kết quả dưới dạng tệp Markdown, CSV và HTML. Yêu cầu phi chức năng gồm ba điểm: chương trình chỉ được dùng thư viện chuẩn của Python ngoài thư viện Tkinter có sẵn, mã nguồn phải được tổ chức thành các mô-đun tách biệt, và phải có bộ kiểm thử tự động kiểm chứng các bất biến quan trọng của thuật toán.

3.2. Kiến trúc phân tầng của chương trình

Chương trình được tổ chức theo kiến trúc phân tầng gồm ba tầng. Tầng trình bày chứa mô-đun giao diện, chịu trách nhiệm dựng cửa sổ, vẽ lưới, xử lý sự kiện chuột và bàn phím, và hiển thị số liệu. Tầng xử lý chứa thư viện tìm kiếm gồm bốn mô-đun con là bộ máy tìm kiếm, biểu diễn lưới, các hàm ước lượng và sinh dữ liệu mẫu. Tầng dữ liệu và kiểm chứng chứa bộ dữ liệu mẫu dưới dạng văn bản và JSON, bộ kiểm thử tự động và kịch bản thực nghiệm.

Cách phân tầng này mang lại hai lợi ích cụ thể. Thứ nhất, tầng xử lý hoàn toàn không phụ thuộc vào Tkinter, do đó cùng một thư viện tìm kiếm có thể được dùng

lại trong kịch bản dòng lệnh và trong bộ kiểm thử tự động mà không cần thay đổi mã nguồn. Thứ hai, việc đo đạc hiệu năng có thể thực hiện ở tầng xử lý nên không bị ảnh hưởng bởi thời gian vẽ của giao diện. Hình 3.1 mô tả ba tầng cùng chiều phụ thuộc giữa chúng.



*Hình 3.1: Kiến trúc phân tầng của chương trình mô phỏng thuật toán A**

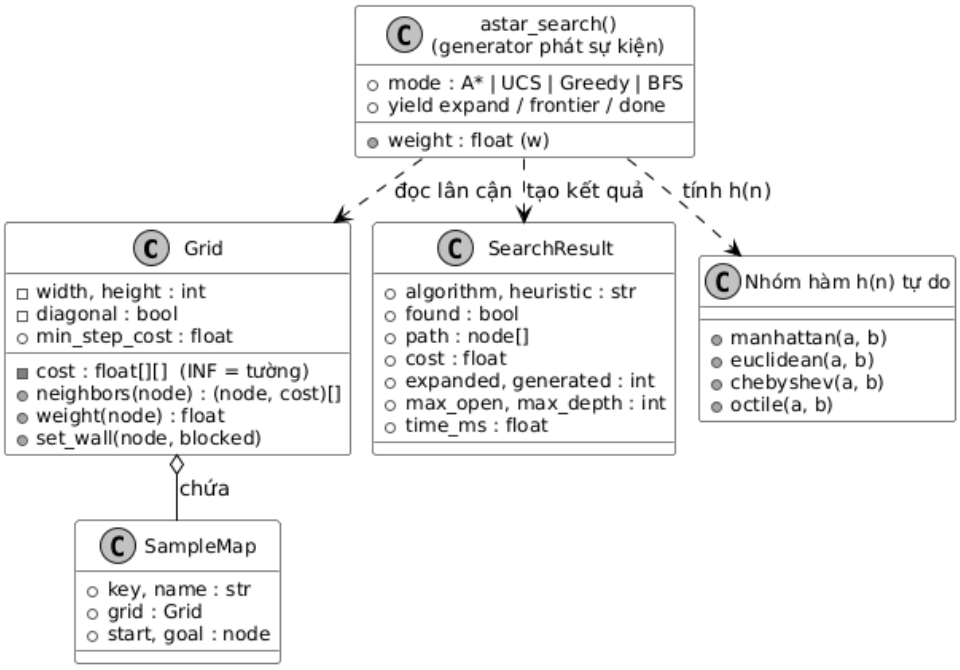
Chiều phụ thuộc giữa các tầng là một chiều. Tầng trình bày gọi tầng xử lý theo từng bước thông qua bộ sinh sự kiện, tầng xử lý đọc dữ liệu lưới và trả về kết quả, tầng kiểm chứng chạy các thí nghiệm trên tầng xử lý và ghi kết quả ra tệp. Không có phụ thuộc ngược từ tầng xử lý lên tầng trình bày, điều này được kiểm chứng bằng việc bộ kiểm thử tự động chạy được mà không cần khởi tạo cửa sổ đồ hoạ.

3.3. Thiết kế lớp và các mô-đun chính

Thư viện tìm kiếm gồm bốn lớp và một nhóm hàm tự do. Lớp Grid biểu diễn lưới với ma trận chi phí, cờ cấu hình bốn hay tám hướng, và bộ đệm danh sách đỉnh kề. Lớp SampleMap gói một bộ dữ liệu mẫu gồm lưới, đỉnh xuất phát, đỉnh đích và phần mô tả. Lớp SearchResult lưu toàn bộ kết quả của một lần tìm kiếm gồm đường đi, tổng chi phí, số đỉnh đã duyệt, số đỉnh đã sinh, kích thước lớn nhất của tập OPEN, độ sâu lớn nhất và thời gian chạy. Nhóm hàm tự do chứa bốn hàm ước lượng và hàm khoảng cách tương ứng.

Bộ máy tìm kiếm được cài đặt dưới dạng một hàm sinh sự kiện. Thay vì chạy trọn vẹn từ đầu đến cuối rồi mới trả về kết quả, hàm này lần lượt phát ra các sự kiện như đỉnh vừa được đưa vào tập OPEN, đỉnh vừa được lấy ra duyệt, đỉnh vừa được

mở lại, và sự kiện kết thúc kèm đối tượng kết quả. Nhờ thiết kế này, giao diện có thể dừng giữa chừng, chạy từng bước hoặc chạy với tốc độ tuỳ chọn mà không cần viết lại thuật toán. Hình 3.2 mô tả quan hệ giữa các lớp và nhóm hàm này.



Hình 3.2: Sơ đồ lớp lõi của thư viện tìm kiếm và quan hệ giữa các thành phần

Bảng 3.1 đặc tả các hàm chính của tầng xử lý. Việc đặc tả rõ kiểu tham số và kiểu trả về giúp phần cài đặt giao diện và phần kiểm thử sử dụng thư viện theo cùng một cách hiểu, đồng thời là cơ sở để mở rộng thêm thuật toán mới mà không phải sửa các mô-đun đã có.

Hàm	**Tham số**	**Kết quả trả về**
search_steps	lưới, xuất phát, đích, thuật toán, hàm ước lượng, trọng số	bộ sinh các sự kiện của quá trình tìm kiếm
search	như trên	đối tượng kết quả sau khi chạy xong toàn bộ
neighbors	một ô của lưới	danh sách các cặp ô kề cùng chi phí bước đi
weight	một ô của lưới	trọng số của ô, bằng vô cùng nếu là tường
from_text, to_text	danh sách dòng ký tự hoặc đối tượng lưới	đối tượng lưới có đỉnh xuất phát, đỉnh đích hoặc chuỗi ký tự
sample_maps	cờ cấu hình bốn hay tám hướng	danh sách sáu bộ dữ liệu mẫu đã dựng

Hàm	Tham số	Kết quả trả về
validate_path	lưới, xuất phát, đích, đường đi	cặp giá trị gồm cờ hợp lệ và thông báo mô tả

Bảng 3.1: Đặc tả các hàm chính của tầng xử lý

3.4. Cấu trúc dữ liệu biểu diễn lưới

Lưới được biểu diễn bằng hai cấu trúc dữ liệu. Cấu trúc thứ nhất là ma trận chi phí hai chiều kích thước H nhân W, trong đó phần tử ở hàng r cột c là trọng số để đi vào ô đó. Ô bị chặn được biểu diễn bằng giá trị vô cùng, nhờ đó phép kiểm tra ô có đi được hay không chỉ là một phép so sánh. Cấu trúc thứ hai là bộ đệm danh sách đỉnh kề dạng bảng băm, lưu kết quả sinh lân cận của từng ô đã từng được hỏi tới. Bộ đệm này bị xoá mỗi khi lưới thay đổi cấu hình hoặc trọng số.

Việc lưu đệm lân cận mang lại lợi ích đáng kể trong các thuật toán có mở lại đỉnh. Khi một đỉnh được sinh kề nhiều lần trong các vòng lặp khác nhau, chương trình không phải tính lại hệ số khoảng cách và kiểm tra cắt góc cho từng hướng, mà chỉ đọc lại danh sách đã tính trước. Chi phí bộ nhớ của bộ đệm là tuyến tính theo số ô đã được hỏi tới, trong khi lợi ích về thời gian thể hiện rõ nhất trên các bộ dữ liệu có nhiều đỉnh được sinh lặp lại.

Để thuận tiện cho việc trao đổi dữ liệu và kiểm thử, lưới còn được biểu diễn dưới dạng văn bản với mỗi dòng tương ứng một hàng và mỗi ký tự tương ứng một ô. Cách biểu diễn này cho phép viết các bộ dữ liệu kiểm thử rất gọn ngay trong mã nguồn, đồng thời cho phép người dùng mở và chỉnh sửa bản đồ bằng bất kì trình soạn thảo văn bản nào. Bảng 3.2 liệt kê bộ ký tự được quy ước.

Ký tự	Ý nghĩa	Trọng số đi vào ô
dấu chấm	ô trống đi được	1
dấu hai chấm	địa hình nhẹ như cỏ hoặc đường mòn	2
dấu ngã	địa hình nặng như cát lún hoặc nước nông	4
dấu bằng	địa hình rất nặng như bùn lầy hoặc dốc đứng	6
dấu thăng	tường, không thể đi vào	vô cùng
chữ S	đỉnh xuất phát	1
chữ G	đỉnh đích	1
dấu hoa thị	ô thuộc đường đi khi kết xuất kết quả	không áp dụng
chữ o	ô đã duyệt xong khi kết xuất kết quả	không áp dụng

Ký tự	Ý nghĩa	Trọng số đi vào ô
dấu gạch nối	ô đang chờ trong tập OPEN khi kết xuất	không áp dụng

Bảng 3.2: Quy ước ký tự trong tệp biểu diễn lưới

3.5. Cấu trúc dữ liệu hàng đợi ưu tiên và các bảng tra cứu

Tập OPEN được cài đặt bằng cây vun đống nhị phân trên một mảng, sử dụng mô-đun hàng đợi ưu tiên của thư viện chuẩn. Mỗi phần tử trong đống là một bộ bốn thành phần gồm giá trị ưu tiên, giá trị ước lượng dùng để phá thế bằng, một số đếm tăng dần và toạ độ của ô. Số đếm tăng dần đóng vai trò quan trọng vì nếu hai phần tử có cùng hai giá trị đầu, phép so sánh sẽ chuyển sang so sánh toạ độ, điều này không gây lỗi nhưng làm mất tính ổn định của thứ tự duyệt. Việc thêm số đếm đảm bảo mọi phần tử đều so sánh được và thứ tự duyệt là xác định giữa các lần chạy.

Bảng giá trị g và bảng cha được cài đặt bằng bảng băm với khoá là toạ độ ô. Bảng giá trị g lưu chi phí tốt nhất đã biết tới mỗi đỉnh đã sinh. Bảng cha lưu đỉnh đứng ngay trước trên đường đi tốt nhất tới đỉnh đó, phục vụ bước truy vết đường đi khi kết thúc. Chương trình còn duy trì một bảng độ sâu để ghi nhận độ sâu lớn nhất của cây tìm kiếm, phục vụ việc phân tích ở Chương 4. Cả ba bảng đều chỉ được ghi cho các đỉnh đã sinh nên bộ nhớ sử dụng tỉ lệ với số đỉnh đã sinh chứ không phải với toàn bộ lưới.

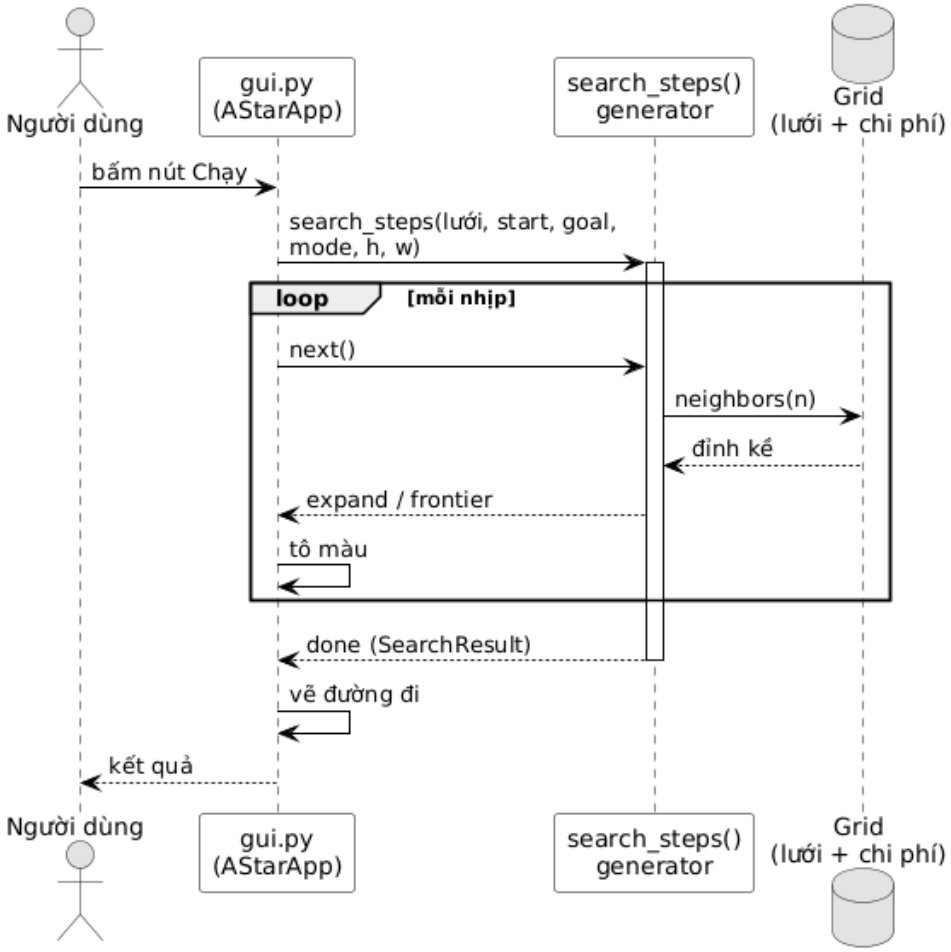
Khi một đỉnh đã nằm trong tập CLOSED được phát hiện có chi phí tốt hơn, chương trình xử lý theo hai chế độ được chọn bởi tham số cấu hình. Chế độ mặc định cho phép mở lại đỉnh, tức là loại đỉnh đó khỏi tập CLOSED và đưa trở lại tập OPEN với chi phí mới. Chế độ thứ hai khoá tập CLOSED, tức là bỏ qua mọi cập nhật cho đỉnh đã duyệt. Chương trình đếm riêng số lần mở lại đỉnh để phục vụ phân tích ở mục 4.8.

3.6. Thiết kế giao diện người dùng

Giao diện được chia thành hai vùng. Vùng bên trái là khung vẽ lưới, mỗi ô được vẽ bằng một hình chữ nhật có màu theo trạng thái hiện tại gồm nền địa hình, tập OPEN, tập CLOSED, đường đi, đỉnh xuất phát, đỉnh đích và đỉnh đang xét. Dưới khung vẽ là thanh trạng thái hiển thị thông tin của ô đang trỏ chuột và thông báo kết quả sau khi chạy xong. Vùng bên phải là bảng điều khiển gồm ba thẻ: thẻ chạy mô phỏng, thẻ bản đồ và thẻ hướng dẫn.

Thẻ chạy mô phỏng gồm bốn nhóm chức năng. Nhóm thứ nhất chọn thuật toán, hàm ước lượng và trọng số, kèm dòng cảnh báo khi tổ hợp được chọn không đảm bảo tính tối ưu. Nhóm thứ hai gồm các nút chạy, dừng, tiến một bước và đặt lại, cùng thanh trượt điều chỉnh tốc độ mô phỏng. Nhóm thứ ba là bảng số liệu hiển thị toạ độ đỉnh đang xét, các giá trị g, h, f, số đỉnh đã duyệt, số đỉnh đã sinh, kích thước hai tập, độ sâu lớn nhất, chi phí đường đi, số ô, thời gian tìm kiếm và số vòng lặp. Nhóm thứ tư gồm hai nút so sánh bốn thuật toán và xuất kết quả ra tệp CSV.

Thẻ bản đồ gồm các công cụ vẽ tường, tô địa hình, đặt đỉnh xuất phát và đặt đỉnh đích, các nút sinh vật cản ngẫu nhiên theo mật độ chọn trước, sinh mê cung, xoá toàn bộ lưới, nạp sáu bộ dữ liệu mẫu, mở và ghi tệp bản đồ, và các nút thay đổi cỡ ô để phóng to hoặc thu nhỏ khung vẽ. Thẻ hướng dẫn giải thích ý nghĩa các màu, liệt kê các phím tắt và nhắc lại định nghĩa của các đại lượng g, h, f để người dùng mới làm quen có thể hiểu ngay ý nghĩa của các con số xuất hiện trên màn hình.



Hình 3.3: Trình tự mô phỏng từng bước giữa giao diện và bộ máy tìm kiếm

Hình 3.3 mô tả trình tự tương tác giữa người dùng, giao diện, bộ máy tìm kiếm và lưới. Mỗi nhịp mô phỏng, giao diện gọi hàm sinh sự kiện để lấy ra một sự kiện, sau đó cập nhật màu của đúng những ô liên quan thay vì vẽ lại toàn bộ khung. Cách cập nhật cục bộ này giữ cho thao tác mô phỏng mượt ngay cả trên lưới có hơn một nghìn tám trăm ô. Khi nhận được sự kiện kết thúc, giao diện vẽ đường đi và nạp đầy đủ số liệu vào bảng thống kê.

3.7. Bộ dữ liệu mẫu và kịch bản thực nghiệm

Sáu bộ dữ liệu mẫu được sinh bởi chính chương trình theo các quy tắc xác định để đảm bảo mọi lần chạy đều cho cùng một kết quả. Bộ dữ liệu lưới trống dùng để quan sát hành vi của thuật toán khi không có vật cản. Bộ dữ liệu vật cản ngẫu nhiên dùng để kiểm thử khả năng vòng tránh với mật độ tường khoảng hai mươi tám phần trăm, có kiểm tra tính liên thông giữa đỉnh xuất phát và đỉnh đích trước khi chấp nhận bản đồ.

Bộ dữ liệu mê cung được sinh bằng thuật toán đào lùi trên lưới có kích thước lẻ, cho ra cấu trúc có duy nhất một đường đi giữa hai ô bất kì. Bộ dữ liệu phòng và hành lang được sinh bằng cách tạo ngẫu nhiên các phòng chữ nhật rồi nối tâm các phòng bằng hành lang chữ L, mô phỏng cấu trúc của một toà nhà hoặc nhà máy. Bộ dữ liệu hành lang xoắn ốc được sinh bằng cách cho một tác nhân đi vòng vào tâm, tạo ra bản đồ chỉ có một lối đi duy nhất. Bộ dữ liệu địa hình trọng số rải ngẫu nhiên các vùng có trọng số hai, bốn và sáu để buộc thuật toán phải cân nhắc giữa đường ngắn và đường rẻ.

Kịch bản thực nghiệm chạy trên tất cả sáu bộ dữ liệu với bốn thuật toán và bốn hàm ước lượng, ghi lại mười tám chỉ số cho mỗi lần chạy. Kết quả được xuất ra ba định dạng gồm tệp CSV để mở bằng bảng tính, tệp Markdown để dán vào báo cáo và tệp HTML có biểu đồ để xem nhanh. Ngoài ra kịch bản còn lưu lại một lần chạy kèm trực quan hoá bằng ký tự để có thể đối chiếu ngay trên dòng lệnh mà không cần mở giao diện đồ hoạ.

CHƯƠNG 4: CÀI ĐẶT, THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Môi trường cài đặt và công cụ sử dụng

Chương trình được cài đặt bằng ngôn ngữ Python phiên bản 3.14 trên hệ điều hành Windows. Toàn bộ thư viện tìm kiếm, sinh dữ liệu mẫu và kiểm thử chỉ sử dụng thư viện chuẩn của Python, bao gồm mô-đun hàng đợi ưu tiên, mô-đun toán học, mô-đun đọc ghi tệp và mô-đun kiểm thử tự động. Giao diện đồ hoạ dùng Tkinter, cũng là thư viện có sẵn trong bản cài đặt chuẩn. Nhờ vậy, chương trình chạy được ngay sau khi cài Python mà không cần bước cài thêm gói phụ thuộc nào.

Việc đo thời gian được thực hiện bằng đồng hồ đơn điệu có độ phân giải cao của mô-đun đo thời gian. Với mỗi lần tìm kiếm, chương trình ghi lại thời điểm bắt đầu trước khi khởi tạo tập OPEN và thời điểm kết thúc ngay sau khi trả về kết quả. Cách đo này chỉ tính thời gian bộ máy tìm kiếm thực sự chạy, không tính thời gian dựng giao diện, do đó số liệu đo được có thể so sánh trực tiếp giữa các thuật toán. Riêng số liệu hiển thị trên giao diện cũng được tính theo nguyên tắc tách riêng phần tìm kiếm và phần vẽ, chi tiết này được trình bày ở mục 4.10.

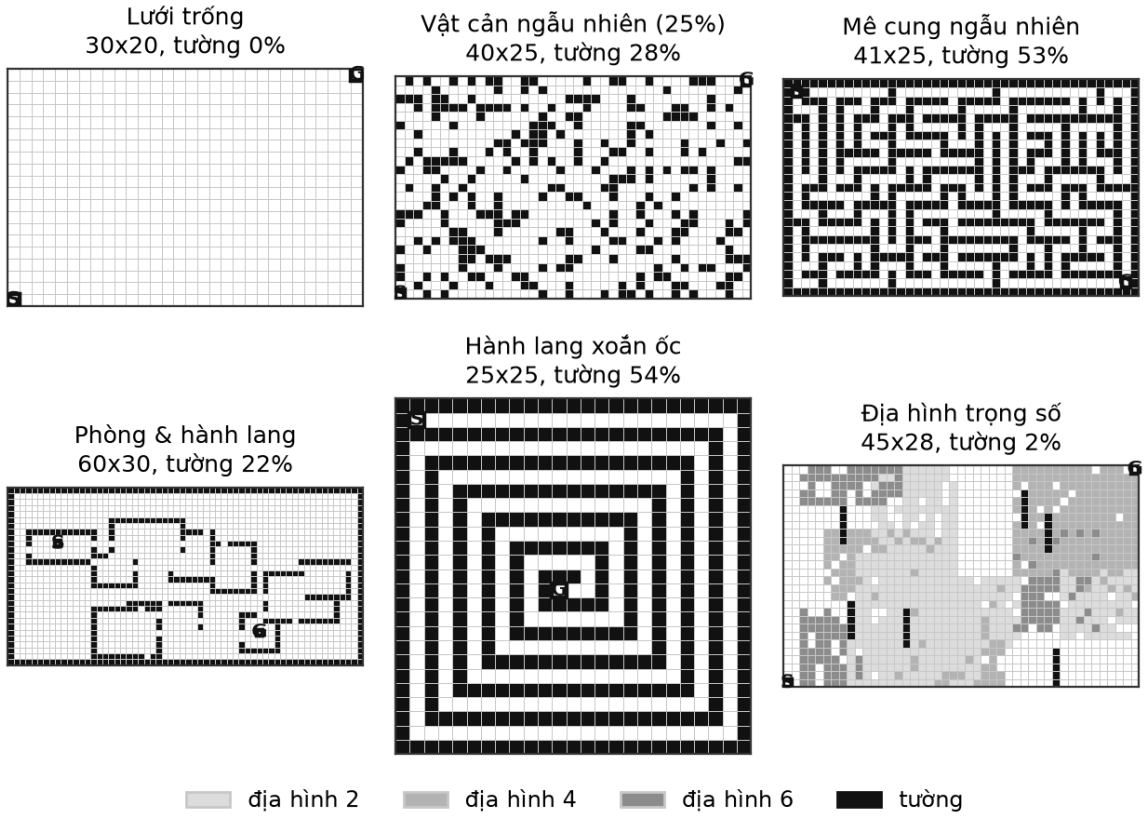
Thành phần	**Thông số**
Hệ điều hành	Windows, kiến trúc 64 bit
Ngôn ngữ	Python 3.14, chạy ở chế độ dòng lệnh và chế độ giao diện
Thư viện dùng cho giao diện	Tkinter có sẵn trong bản cài đặt chuẩn
Thư viện dùng cho thực nghiệm	Chỉ thư viện chuẩn, không dùng gói ngoài
Kích thước lưới lớn nhất	60 hàng nhân 30 cột, tương đương 1800 ô
Số bộ dữ liệu mẫu	6 bộ, sinh xác định theo hạt giống ngẫu nhiên
Số ca kiểm thử tự động	28 ca, chạy trong khoảng 0,2 giây
Định dạng kết xuất	CSV, JSON, Markdown, HTML, ảnh PNG

Bảng 4.1: Môi trường cài đặt và cấu hình thử nghiệm

4.2. Sáu bộ dữ liệu mẫu

Sáu bộ dữ liệu mẫu được chọn để bao phủ các cấu trúc bản đồ khác nhau, từ trường hợp không có vật cản tới trường hợp lưới chỉ có một lối đi duy nhất. Hình 4.1 trình bày hình dạng của cả sáu bộ dữ liệu cùng tỉ lệ ô tường của mỗi bộ. Việc

đặt chúng cạnh nhau cho thấy rõ sự khác biệt về cấu trúc và giải thích vì sao cùng một thuật toán lại có kết quả rất khác nhau giữa các bộ dữ liệu.



Hình 4.1: Sáu bộ dữ liệu mẫu dùng cho thực nghiệm, kèm kích thước và tỉ lệ tường

Bộ dữ liệu lưới trống có kích thước ba mươi nhân hai mươi ô và không có tường. Bộ dữ liệu vật cản ngẫu nhiên có kích thước bốn mươi nhân hai mươi lăm ô với khoảng hai mươi tám phần trăm ô là tường. Bộ dữ liệu mê cung có kích thước bốn mươi mốt nhân hai mươi lăm ô, tỉ lệ tường hơn năm mươi phần trăm và là bản đồ liên thông yếu, nghĩa là giữa hai ô bất kì chỉ có đúng một đường đi. Bộ dữ liệu phòng và hành lang có kích thước sáu mươi nhân ba mươi ô, gồm mười phòng lớn nối với nhau bằng cửa hẹp. Bộ dữ liệu hành lang xoắn ốc có kích thước hai mươi lăm nhân hai mươi lăm ô và chỉ có một hành lang duy nhất đi từ ngoài vào tâm. Bộ dữ liệu địa hình trọng số có kích thước bốn mươi lăm nhân hai mươi tám ô với các vùng chi phí hai, bốn và sáu rải rác. Bảng 4.2 tổng hợp thông số của sáu bộ dữ liệu.

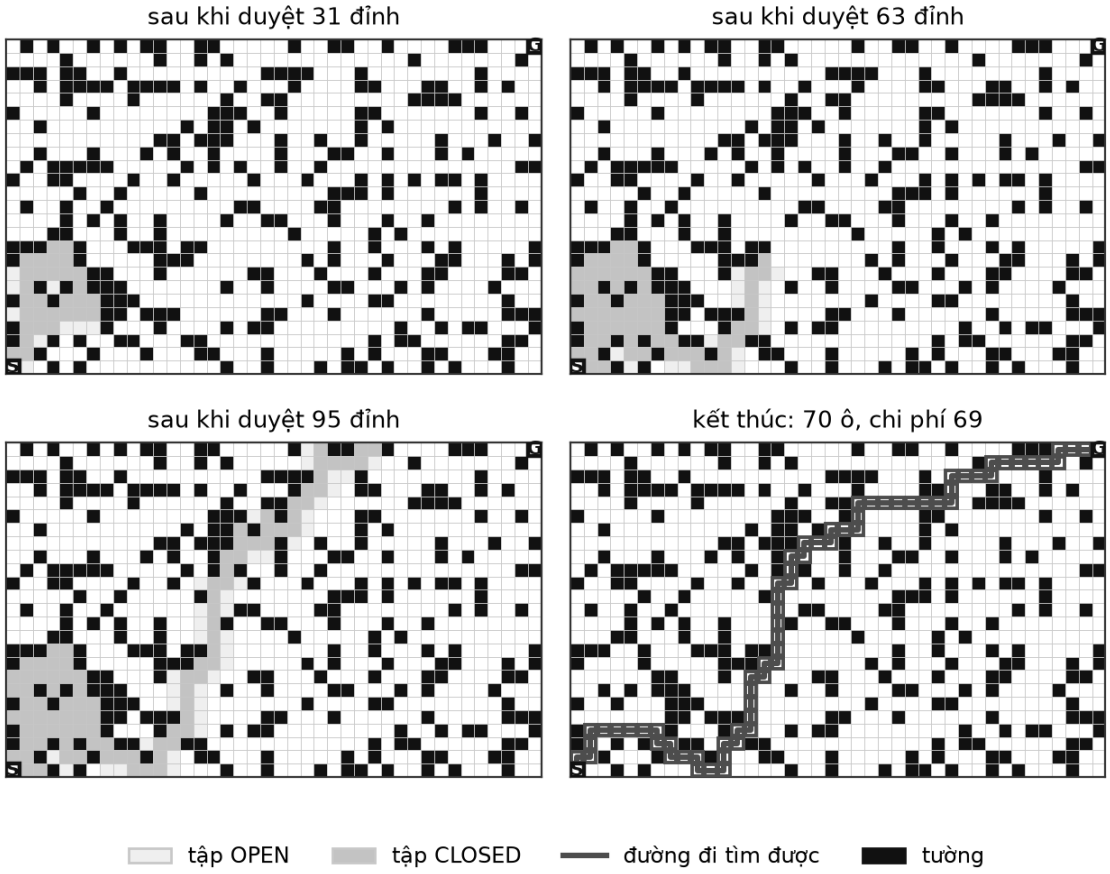
Bộ dữ liệu	**Kích thước**	**Tỉ lệ tường**	**Xuất phát**	**Đích**	**Số ô đi được**
Lưới trống	30x20	0%	(19, 0)	(0, 29)	600

Bộ dữ liệu	Kích thước	Tỉ lệ tường	Xuất phát	Đích	Số ô đi được
Vật cản ngẫu nhiên (25%)	40x25	28%	(24, 0)	(0, 39)	724
Mê cung ngẫu nhiên	41x25	53%	(1, 1)	(23, 39)	479
Phòng & hành lang	60x30	22%	(9, 8)	(24, 42)	1405
Hành lang xoắn ốc	25x25	54%	(1, 1)	(13, 11)	287
Địa hình trọng số	45x28	2%	(27, 0)	(0, 44)	1230

Bảng 4.2: Thông số của sáu bộ dữ liệu mẫu

4.3. Diễn biến quá trình tìm kiếm trên lưới vật cản

Hình 4.2 ghi lại bốn thời điểm của cùng một lần chạy A* trên bộ dữ liệu vật cản ngẫu nhiên. Ở thời điểm đầu, tập CLOSED chỉ là một vùng nhỏ bao quanh đỉnh xuất phát, còn tập OPEN là lớp ô mỏng nằm ở rìa vùng đó. Khi quá trình tiếp diễn, vùng đã duyệt mở rộng thành một dải hẹp uốn theo hướng đích thay vì lan đều ra bốn phía. Ở thời điểm thứ ba, dải đã duyệt gần chạm tới đích. Bức ảnh cuối cùng là kết quả sau khi tìm xong, với đường đi gồm bảy mươi ô và tổng chi phí bằng sáu mươi chín.

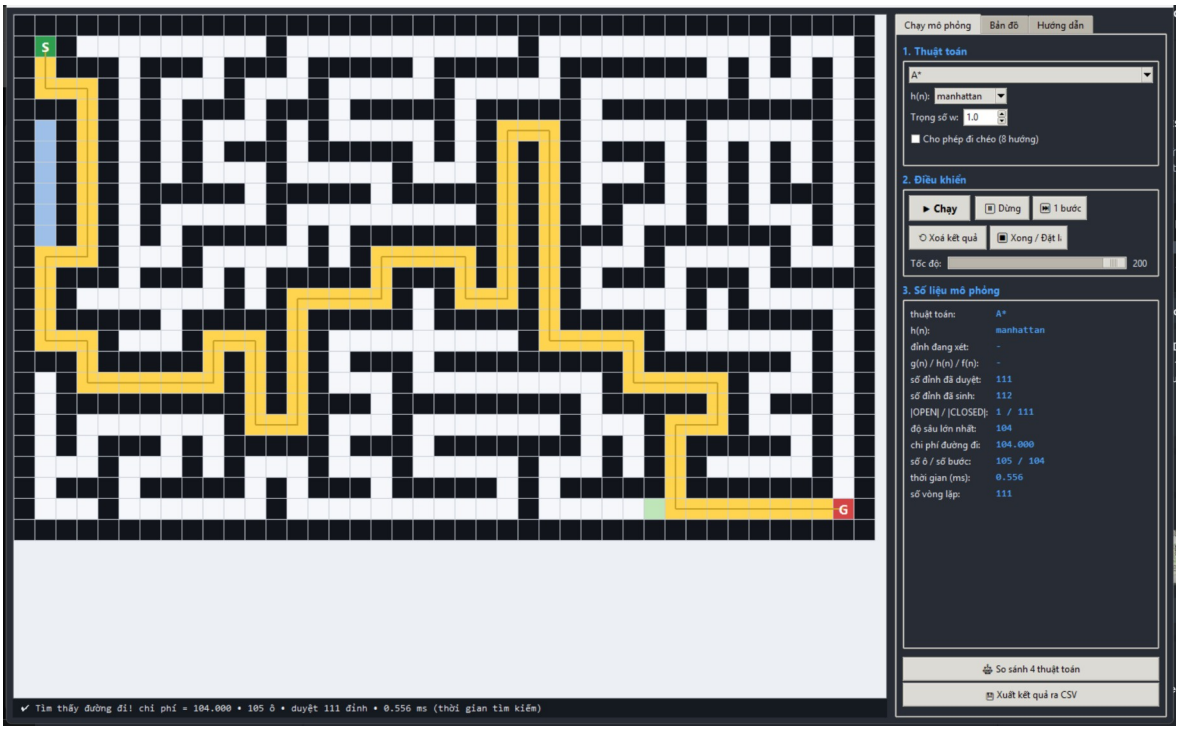


Hình 4.2: Diễn biến quá trình A trên lưới vật cản ngẫu nhiên qua bốn thời điểm*

Điều đáng chú ý trong Hình 4.2 là hình dạng của vùng đã duyệt. Nếu dùng tìm kiếm theo chi phí đều, vùng này sẽ là một hình tròn bán kính tăng dần cho tới khi chạm đích, tức là số đỉnh duyệt tỉ lệ với diện tích hình tròn. Với A*, vùng đã duyệt co lại thành một dải hẹp chạy theo hướng từ đỉnh xuất phát tới đỉnh đích. Chính hình dạng dải hẹp này là nguyên nhân trực tiếp giúp số đỉnh duyệt giảm từ sáu trăm chín mươi ba xuống còn một trăm hai mươi bảy trên cùng một bộ dữ liệu, tương ứng mức giảm khoảng tám mươi hai phần trăm.

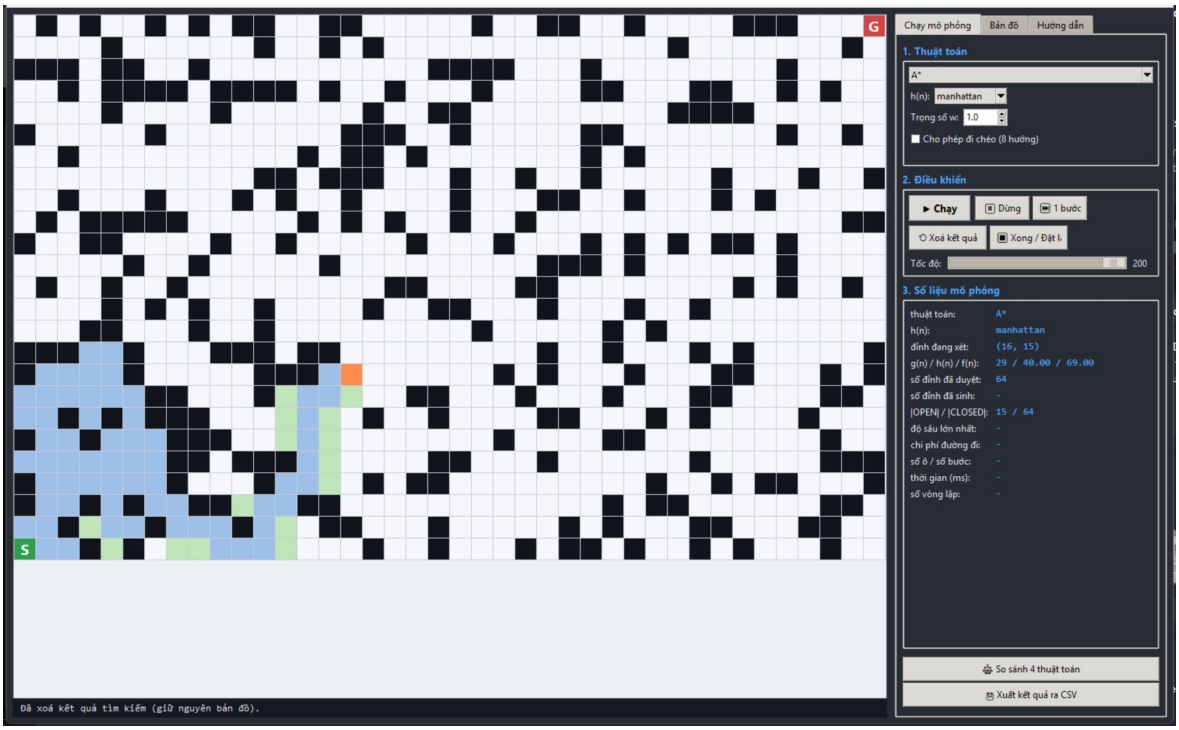
4.4. Giao diện chương trình

Hình 4.3 là ảnh chụp màn hình của giao diện sau khi chạy xong thuật toán A* trên bộ dữ liệu mê cung. Bảng số liệu bên phải hiển thị số đỉnh đã duyệt là một trăm mười một, số đỉnh đã sinh là một trăm mười hai, chi phí đường đi bằng một trăm linh bốn, đường đi gồm một trăm linh năm ô và thời gian tìm kiếm dưới mười mili giây. Thanh trạng thái ở đáy khung vẽ nhắc lại kết quả này bằng một dòng thông báo ngắn.



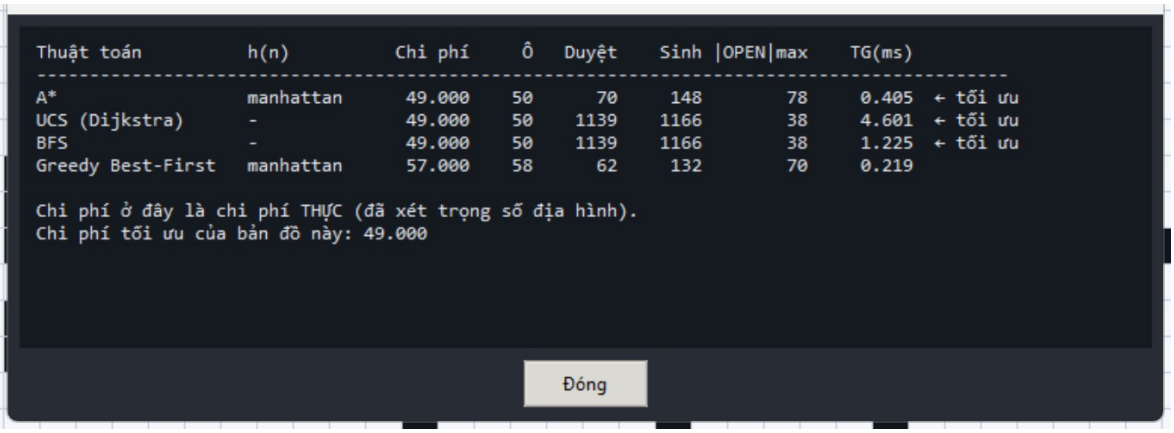
Hình 4.3: Giao diện chương trình sau khi hoàn tất tìm kiếm trên bộ dữ liệu mê cung

Hình 4.4 là ảnh chụp màn hình tại thời điểm quá trình tìm kiếm chưa kết thúc. Ở trạng thái giữa chừng này, vùng màu tối phía dưới bên trái là tập CLOSED đã duyệt, viền xám nhạt là tập OPEN đang chờ, và ô viền đậm là đỉnh đang được xét có toạ độ hàng mười sáu cột mười lăm. Bảng số liệu hiển thị tương ứng giá trị g bằng hai mươi chín, giá trị h bằng bốn mươi và giá trị f bằng sáu mươi chín, cùng số đỉnh đã duyệt là sáu mươi bốn. Đây là hình ảnh minh hoạ trực tiếp cho cách A* mở rộng vùng tìm kiếm có định hướng.



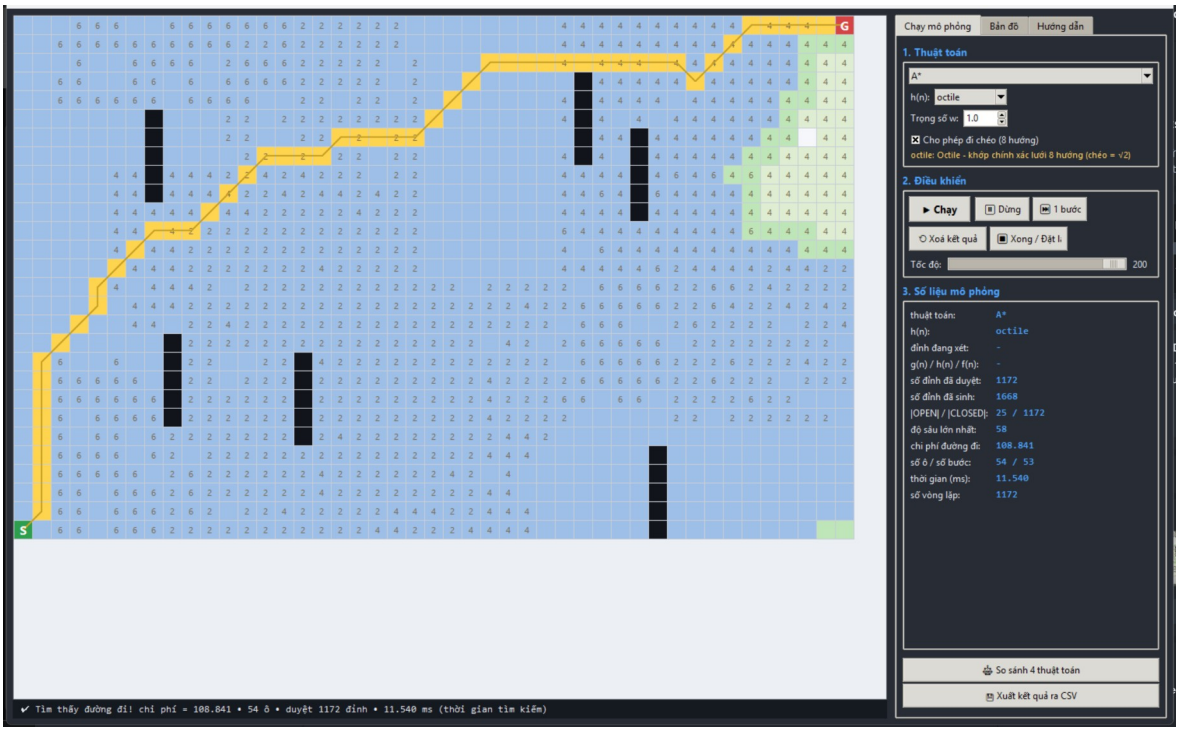
Hình 4.4: Trạng thái giữa chừng của quá trình tìm kiếm với hai tập OPEN và CLOSED

Hình 4.5 là hộp thoại so sánh bốn thuật toán trên bộ dữ liệu phòng và hành lang. Bảng trong hộp thoại liệt kê chi phí thực của đường đi, số ô, số đỉnh đã duyệt, số đỉnh đã sinh, kích thước lớn nhất của tập OPEN và thời gian chạy của từng thuật toán. Ba thuật toán A*, tìm kiếm theo chi phí đều và tìm kiếm theo chiều rộng cho cùng chi phí bằng bốn mươi chín và được đánh dấu là tối ưu, trong khi tìm kiếm tham lam cho chi phí năm mươi bảy, tức là kém hơn mức tối ưu khoảng mười sáu phần trăm.



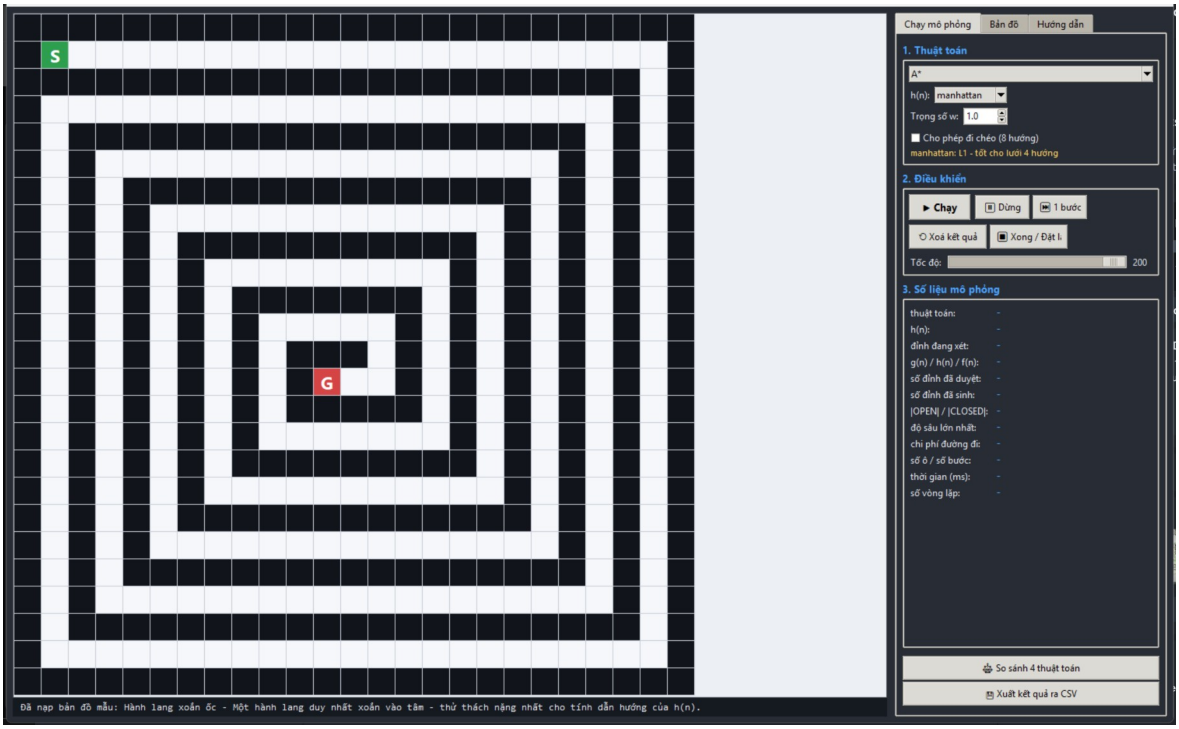
Hình 4.5: Hộp thoại so sánh bốn thuật toán trên cùng một bản đồ

Hình 4.6 minh hoạ bộ dữ liệu địa hình trọng số khi bật cấu hình tám hướng. Các ô có chữ số là ô địa hình nặng với chi phí tương ứng hai, bốn hoặc sáu. Bảng số liệu cho thấy chi phí đường đi bằng một trăm linh tám phẩy tám bốn mươi mốt, số ô trên đường đi là năm mươi bốn và thời gian tìm kiếm lớn hơn đáng kể so với các bộ dữ liệu khác. Việc cho phép đi chéo giúp đường đi ngắn hơn về số ô nhưng đồng thời làm tăng số đỉnh cần duyệt vì mỗi ô có tới tám ô kề.



Hình 4.6: Bộ dữ liệu địa hình có trọng số với cấu hình tám hướng

Hình 4.7 là thẻ hướng dẫn của giao diện, trong đó giải thích ý nghĩa các màu và liệt kê các phím tắt. Việc đưa phần giải thích vào ngay trong chương trình giúp người dùng lần đầu sử dụng có thể hiểu đúng ý nghĩa của từng màu và từng con số mà không cần đọc tài liệu riêng. Bảng 4.3 tóm tắt các chỉ số mà giao diện hiển thị trong quá trình mô phỏng.



Hình 4.7: Thẻ hướng dẫn và chú thích màu trong giao diện chương trình

4.5. Kịch bản kiểm thử và kết quả kiểm thử

Bộ kiểm thử tự động gồm hai mươi tám ca, chia thành bảy nhóm. Nhóm thứ nhất kiểm tra các trường hợp cơ bản gồm đỉnh xuất phát trùng đỉnh đích, đích bị bao kín nên không có đường đi, đỉnh xuất phát hoặc đỉnh đích nằm trên tường, và trường hợp lưới trống yêu cầu chi phí đúng bằng khoảng cách Manhattan. Nhóm thứ hai kiểm tra tính tối ưu bằng cách so sánh chi phí của A* với chi phí của tìm kiếm theo chi phí đều trên cả sáu bộ dữ liệu với hai cấu hình bốn hướng và tám hướng.

Nhóm thứ ba kiểm tra tính hợp lệ của đường đi gồm các điều kiện đường đi phải bắt đầu từ đúng đỉnh xuất phát, kết thúc tại đúng đích, không chứa ô lặp, mọi cặp ô liên tiếp phải kề nhau và không đi xuyên qua tường. Nhóm thứ tư kiểm tra các hàm ước lượng gồm điều kiện giá trị tại đích bằng không và điều kiện không ước lượng vượt quá chi phí tối ưu thực tế trên cả sáu bộ dữ liệu. Nhóm thứ năm kiểm tra các thuật toán đối chứng gồm việc tìm kiếm tham lam không bao giờ cho chi phí tốt hơn mức tối ưu và tìm kiếm theo chiều rộng cho số bước đúng bằng số bước của A* trên lưới không có trọng số.

Nhóm thứ sáu kiểm tra dòng sự kiện của bộ máy tìm kiếm gồm việc số sự kiện duyệt đỉnh phải khớp với số đỉnh đã duyệt trong kết quả, và không có sự kiện nào trỏ tới ô là tường. Nhóm thứ bảy kiểm tra cấu trúc dữ liệu lưới gồm việc biểu diễn văn bản phải đọc lại đúng như đã ghi, số đỉnh kề của ô ở giữa lưới phải bằng bốn hoặc tám tuỳ cấu hình, quy tắc không cắt góc phải chặn được bước đi chéo xuyên qua khe hở giữa hai tường, và chi phí bước đi phải nhân đúng theo trọng số của ô đích. Bảng 4.4 trình bày các kịch bản kiểm thử tiêu biểu và Bảng 4.5 tổng hợp kết quả.

Mã	**Kịch bản kiểm thử**	**Kết quả mong đợi**
TC-01	Đỉnh xuất phát trùng đỉnh đích	Đường đi gồm một ô, chi phí bằng 0
TC-02	Đích bị bốn tường bao kín	Kết luận không tồn tại đường đi
TC-03	Đỉnh xuất phát hoặc đích nằm trên tường	Không tìm thấy đường đi, không báo lỗi
TC-04	Tường chắn giữa hai đỉnh, buộc phải đi vòng	Đường đi hợp lệ và có chi phí nhỏ nhất
TC-05	Lưới trống có trọng số bằng 1	Chi phí đúng bằng khoảng cách Manhattan
TC-06	Đường đi chéo trên lưới tám hướng	Tổng chi phí bằng ba lần căn bậc hai của hai
TC-07	Lưới địa hình trọng số, so với chi phí đều	Chi phí của A* bằng chi phí tối ưu
TC-08	Tìm kiếm theo chiều rộng trên lưới có trọng số	Chi phí thực của đường đi không nhỏ hơn mức tối ưu
TC-09	Kiểm tra đường đi không xuyên qua tường	Mọi cặp ô liên tiếp đều kề nhau và hợp lệ
TC-10	Hàm ước lượng tại đích	Giá trị bằng 0 với cả bốn hàm
TC-11	Kiểm tra số sự kiện trùng với số đỉnh đã duyệt	Hai giá trị bằng nhau
TC-12	Ghi lưới ra văn bản rồi đọc lại	Lưới, xuất phát và đích giữ nguyên

Bảng 4.4: Kịch bản kiểm thử tiêu biểu

Nhóm kiểm thử	**Số ca**	**Kết quả**
Nhóm 1: trường hợp cơ bản và biên	6	Đạt 6/6
Nhóm 2: tính tối ưu so với chi phí đều	5	Đạt 5/5
Nhóm 3: tính hợp lệ của đường đi	2	Đạt 2/2
Nhóm 4: tính khả dụng của hàm ước lượng	4	Đạt 4/4
Nhóm 5: thuật toán đối chứng	4	Đạt 4/4
Nhóm 6: dòng sự kiện của bộ máy tìm kiếm	2	Đạt 2/2

Nhóm kiểm thử	Số ca	Kết quả
Nhóm 7: cấu trúc dữ liệu lưới	5	Đạt 5/5
Tổng cộng	28	Đạt 28/28

Bảng 4.5: Tổng hợp kết quả kiểm thử theo nhóm

4.6. Thí nghiệm 1 và 2: so sánh A* với các thuật toán đối chứng

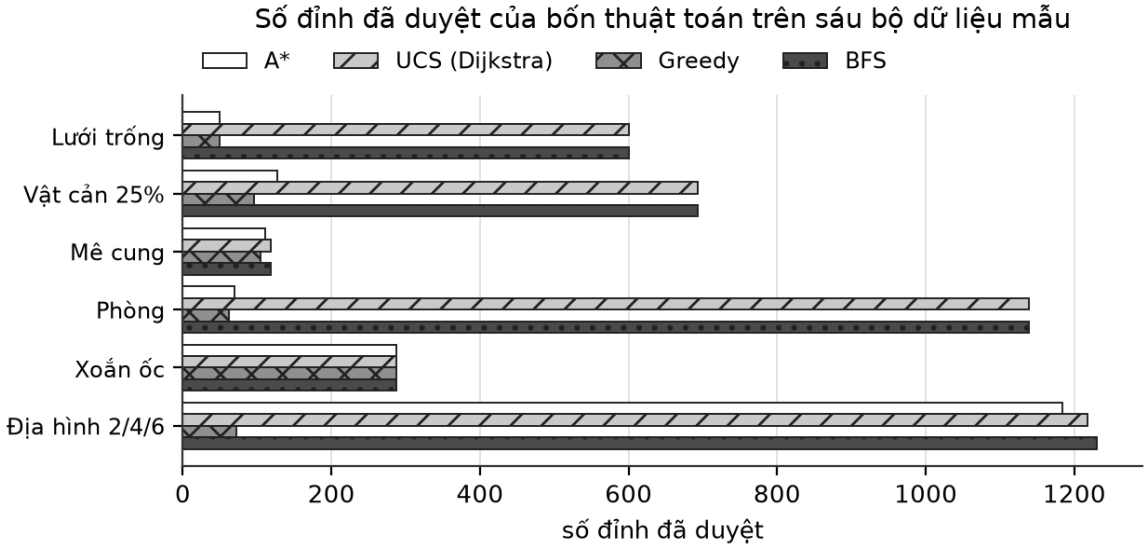
Thí nghiệm thứ nhất chạy A* với bốn hàm ước lượng khác nhau trên cả sáu bộ dữ liệu mẫu. Thí nghiệm thứ hai chạy thêm ba thuật toán đối chứng gồm tìm kiếm theo chi phí đều, tìm kiếm tham lam và tìm kiếm theo chiều rộng trên cùng bộ dữ liệu. Tất cả các lần chạy đều dùng cấu hình bốn hướng, hàm ước lượng Manhattan cho A* và tìm kiếm tham lam, và trọng số bằng một. Chi phí ghi trong bảng là chi phí thực của đường đi, đã tính đầy đủ trọng số địa hình, do đó có thể dùng để so sánh trực tiếp giữa các thuật toán.

Kết quả đầy đủ của hai thí nghiệm được trình bày trong Bảng 4.6 và được biểu diễn bằng biểu đồ ở Hình 4.8. Cột tối ưu cho biết đường đi tìm được có chi phí đúng bằng chi phí tối ưu tham chiếu hay không, trong đó chi phí tham chiếu được lấy từ tìm kiếm theo chi phí đều. Kết quả cho thấy A* luôn đạt mức tối ưu, tìm kiếm theo chi phí đều và tìm kiếm theo chiều rộng cũng đạt mức tối ưu về chi phí thực trên mọi bộ dữ liệu, còn tìm kiếm tham lam không đạt mức tối ưu trên ba bộ dữ liệu.

Bộ dữ liệu	Thuật toán	h(n)	Chi phí	Ô	Duyệt	Sinh	OPEN	TG (ms)	Tối ưu
Lưới trống	A*	manh.	48.000	49	49	95	46	0.203	có
Lưới trống	UCS (Dijkstra)	-	48.000	49	600	600	19	1.345	có
Lưới trống	Greedy Best-First	manh.	48.000	49	49	95	46	0.165	có
Lưới trống	BFS	-	48.000	49	600	600	19	0.803	có
Vật cản 25%	A*	manh.	69.000	70	127	189	54	0.393	có
Vật cản 25%	UCS (Dijkstra)	-	69.000	70	693	693	30	1.505	có
Vật cản 25%	Greedy Best-First	manh.	81.000	82	96	150	54	0.293	không
Vật cản 25%	BFS	-	69.000	70	693	693	30	0.881	có

Bộ dữ liệu	Thuật toán	h(n)	Chi phí	Ô	Duyệt	Sinh	OPEN	TG (ms)	Tối ưu
Mê cung	A*	manh.	104.000	105	111	112	1	0.237	có
Mê cung	UCS (Dijkstra)	-	104.000	105	119	120	1	0.223	có
Mê cung	Greedy Best-First	manh.	104.000	105	105	107	2	0.219	có
Mê cung	BFS	-	104.000	105	119	120	1	0.141	có
Phòng	A*	manh.	49.000	50	70	148	78	0.295	có
Phòng	UCS (Dijkstra)	-	49.000	50	1139	1166	38	2.728	có
Phòng	Greedy Best-First	manh.	57.000	58	62	132	70	0.242	không
Phòng	BFS	-	49.000	50	1139	1166	38	1.543	có
Xoắn ốc	A*	manh.	286.000	287	287	287	0	0.570	có
Xoắn ốc	UCS (Dijkstra)	-	286.000	287	287	287	0	0.520	có
Xoắn ốc	Greedy Best-First	manh.	286.000	287	287	287	0	0.536	có
Xoắn ốc	BFS	-	286.000	287	287	287	0	0.329	có
Địa hình	A*	manh.	128.000	72	1184	1483	181	3.742	có
Địa hình	UCS (Dijkstra)	-	128.000	72	1218	1227	43	2.943	có
Địa hình	Greedy Best-First	manh.	166.000	72	72	141	69	0.256	không
Địa hình	BFS	-	166.000	72	1230	1230	27	1.728	không

Bảng 4.6: Kết quả thí nghiệm 1 và 2 trên sáu bộ dữ liệu mẫu



Hình 4.8: Số đỉnh đã duyệt của bốn thuật toán trên sáu bộ dữ liệu mẫu

Trên bộ dữ liệu lưới trống, A* chỉ duyệt 49 đỉnh trong khi tìm kiếm theo chi phí đều phải duyệt 600 đỉnh, tương ứng mức giảm hơn mười hai lần. Trên bộ dữ liệu vật cản ngẫu nhiên, mức giảm là từ 693 đỉnh xuống 127 đỉnh. Trên bộ dữ liệu phòng và hành lang, mức giảm lớn nhất trong cả sáu bộ dữ liệu khi số đỉnh duyệt giảm từ 1139 xuống 70 đỉnh. Nguyên nhân là cấu trúc phòng lớn nối bằng cửa hẹp khiến tìm kiếm không có thông tin phải lan ra toàn bộ các phòng, trong khi hàm ước lượng giúp A* đi thẳng tới phòng chứa đích.

Ngược lại, trên bộ dữ liệu hành lang xoắn ốc, cả bốn thuật toán đều duyệt 287 đỉnh, đúng bằng số ô đi được của bản đồ. Kết quả này không phải là sai sót mà là hệ quả tất yếu của cấu trúc bản đồ: khi chỉ có một lối đi duy nhất, mọi thuật toán đều buộc phải đi hết hành lang nên không có chỗ cho việc cắt giảm. Đây là ví dụ rõ ràng cho thấy hiệu quả của hàm ước lượng phụ thuộc vào cấu trúc của bài toán.

Trường hợp tìm kiếm tham lam cũng rất đáng chú ý. Thuật toán này duyệt ít đỉnh nhất trên nhiều bộ dữ liệu, chẳng hạn chỉ 62 đỉnh trên bộ dữ liệu phòng và hành lang, nhưng chi phí đường đi lại là 57 so với mức tối ưu 49. Trên bộ dữ liệu địa hình trọng số, mức chênh lệch còn lớn hơn khi chi phí của tìm kiếm tham lam là 166 so với 128. Số đỉnh duyệt ít không đồng nghĩa với chất lượng đường đi tốt, và đây chính là lí do vì sao tìm kiếm tham lam không được dùng trong các ứng dụng đòi hỏi đường đi rẻ nhất.

Một quan sát khác liên quan tới tìm kiếm theo chiều rộng. Trên bộ dữ liệu địa hình trọng số, thuật toán này cho đường đi có chi phí thực bằng 166, cao hơn mức tối ưu khoảng ba mươi phần trăm. Nguyên nhân là tìm kiếm theo chiều rộng coi mọi bước đi có cùng chi phí một và bỏ qua hoàn toàn trọng số địa hình, do đó nó ưu tiên đường có ít bước nhất dù đường đó phải đi xuyên qua vùng cát hoặc bùn. Kết quả này cho thấy tính tối ưu của một thuật toán chỉ có ý nghĩa trong khuôn khổ mô hình bài toán mà nó giả định.

Bộ dữ liệu	**A***	**Chi phí đều**	**Mức giảm**	**Chi phí A***	**Chi phí đều**
Lưới trống	49	600	91.8%	48.000	48.000
Vật cản 25%	127	693	81.7%	69.000	69.000
Mê cung	111	119	6.7%	104.000	104.000
Phòng	70	1139	93.9%	49.000	49.000
Xoắn ốc	287	287	0.0%	286.000	286.000
Địa hình	1184	1218	2.8%	128.000	128.000

Bảng 4.7: Mức giảm số đỉnh duyệt của A so với tìm kiếm theo chi phí đều*

4.7. Thí nghiệm 3: ảnh hưởng của hàm ước lượng

Thí nghiệm thứ ba giữ nguyên thuật toán A* và lần lượt thay hàm ước lượng bằng khoảng cách Manhattan, Euclid, Chebyshev và octile, đo số đỉnh phải duyệt trên bốn bộ dữ liệu đại diện. Mục đích của thí nghiệm là định lượng ảnh hưởng của độ chặt hàm ước lượng tới khối lượng công việc mà thuật toán phải thực hiện, đồng thời kiểm chứng nhận xét lý thuyết ở mục 2.6.

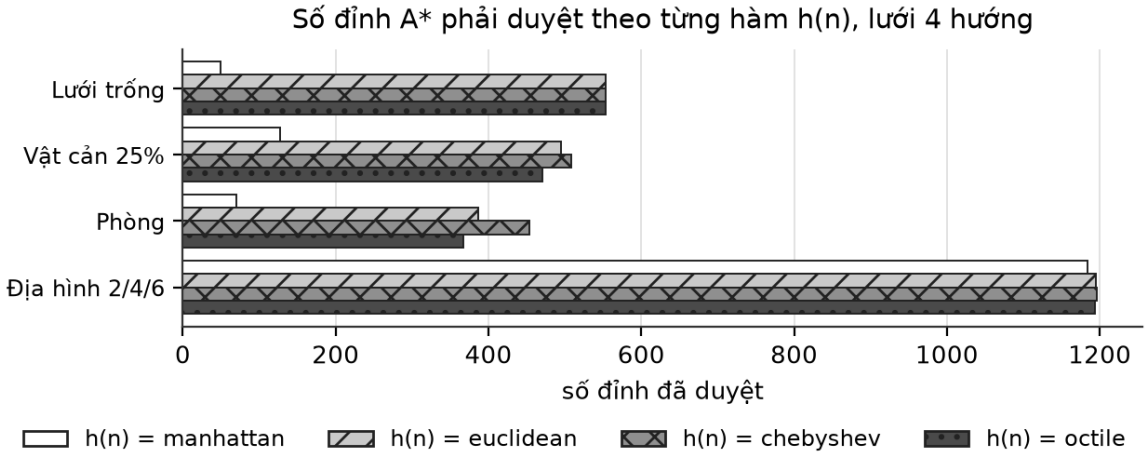
Kết quả được trình bày trong Bảng 4.8 và Hình 4.9. Trên lưới trống, khoảng cách Manhattan chỉ cần bốn mươi chín đỉnh duyệt trong khi ba hàm còn lại cần hơn năm trăm đỉnh. Nguyên nhân là khoảng cách Manhattan khớp chính xác với chi phí thực trên lưới bốn hướng không có vật cản, do đó A* chỉ cần duyệt đúng các đỉnh nằm trên đường đi tối ưu. Khoảng cách Euclid và Chebyshev thấp hơn chi phí thực trong trường hợp này nên đỉnh được ưu tiên theo hướng khác, khiến thuật toán phải duyệt thêm nhiều đỉnh không cần thiết.

Trên các bộ dữ liệu có vật cản, khoảng cách giữa các hàm ước lượng thu hẹp đáng kể vì cả bốn hàm đều trở nên lỏng hơn so với chi phí thực. Trên bộ dữ liệu vật cản ngẫu nhiên, số đỉnh duyệt của bốn hàm lần lượt là một trăm hai mươi bảy, bốn

trăm chín mươi lăm, năm trăm linh tám và bốn trăm bảy mươi, tức là khoảng cách Manhattan vẫn tốt nhất nhưng mức chênh lệch chỉ còn khoảng bốn lần. Trên bộ dữ liệu địa hình trọng số, bốn hàm cho kết quả gần như bằng nhau vì trọng số địa hình lớn làm cho mọi ước lượng đều nhỏ hơn nhiều so với chi phí thực.

Bộ dữ liệu	**Manhattan**	**Euclid**	**Chebyshev**	**Octile**
Lưới trống	49	553	553	553
Vật cản 25%	127	495	508	470
Phòng	70	386	454	367
Địa hình	1184	1195	1196	1194

Bảng 4.8: Số đỉnh A phải duyệt theo từng hàm ước lượng*



Hình 4.9: Số đỉnh A phải duyệt theo từng hàm ước lượng, lưới 4 hướng*

Kết luận rút ra từ thí nghiệm này gồm hai điểm. Thứ nhất, khi cấu hình di chuyển đã biết, hàm ước lượng khớp chính xác với chi phí thực là lựa chọn tốt nhất, do đó với lưới bốn hướng nên dùng khoảng cách Manhattan và với lưới tám hướng nên dùng khoảng cách octile. Thứ hai, khi bản đồ có nhiều vật cản hoặc có địa hình trọng số lớn, chênh lệch giữa các hàm ước lượng thu hẹp lại, nghĩa là lợi ích của việc chọn hàm ước lượng chặt không còn quyết định như trong trường hợp bản đồ thoáng.

4.8. Thí nghiệm 4: biến thể A* có trọng số

Thí nghiệm thứ tư khảo sát biến thể A* có trọng số bằng cách nhân thêm hệ số w lần lượt bằng một, một phẩy năm, hai và ba vào hàm ước lượng. Thí nghiệm được chạy trên bộ dữ liệu địa hình trọng số, là bộ dữ liệu bộc lộ rõ nhất sự đánh đổi

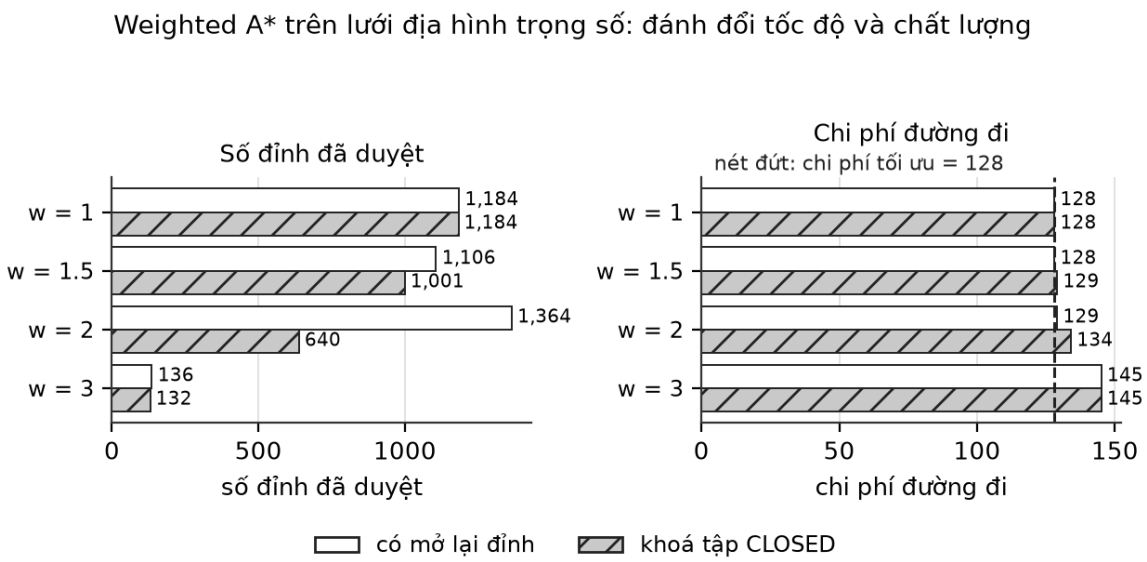
giữa số đỉnh phải duyệt và chất lượng đường đi. Mỗi giá trị của w được chạy ở hai chế độ, chế độ cho phép mở lại đỉnh đã duyệt và chế độ khoá tập CLOSED.

Kết quả trong Bảng 4.9 và Hình 4.10 cho thấy một hiện tượng thú vị. Khi w tăng lên một phẩy năm, số đỉnh duyệt giảm nhẹ từ một nghìn một trăm tám mươi bốn xuống một nghìn một trăm lẻ sáu đỉnh và chi phí đường đi vẫn giữ nguyên ở mức tối ưu. Khi w tăng lên hai và vẫn bật chế độ mở lại đỉnh, số đỉnh duyệt lại tăng vọt lên một nghìn ba trăm sáu mươi bốn, cao hơn cả trường hợp w bằng một. Nguyên nhân là giá trị ưu tiên không còn đơn điệu, khiến nhiều đỉnh bị mở lại và phải xét lại, làm phát sinh thêm công việc thay vì tiết kiệm công việc.

Khi chuyển sang chế độ khoá tập CLOSED, bức tranh thay đổi hoàn toàn. Số đỉnh duyệt giảm mạnh và đơn điệu theo w, cụ thể là một nghìn một trăm tám mươi bốn, một nghìn không trăm lẻ một, sáu trăm bốn mươi và một trăm ba mươi sáu đỉnh cho bốn giá trị của w. Đổi lại, chi phí đường đi tăng dần và vượt khỏi mức tối ưu: ở w bằng hai chi phí là một trăm ba mươi bốn, ở w bằng ba chi phí là một trăm bốn mươi lăm, tức là cao hơn mức tối ưu một trăm hai mươi tám khoảng mười ba phần trăm.

Bộ dữ liệu	**w**	**Mở lại**	**Chi phí**	**Ô**	**Duyệt**	**Sinh**	**OPEN**	**TG (ms)**	**Tối ưu**
Địa hình	1	có	128.000	72	1184	1483	181	3.742	có
Địa hình	1	có	128.000	72	1195	1258	80	3.257	có
Địa hình	1	có	128.000	72	1196	1304	77	3.453	có
Địa hình	1	có	128.000	72	1194	1257	88	3.391	có
Địa hình	1	có	128.000	72	1184	1483	181	3.767	có
Địa hình	1.5	có	128.000	72	1106	1558	174	3.806	có
Địa hình	2	có	129.000	74	1364	1956	241	5.041	không
Địa hình	3	có	145.000	72	136	245	103	0.484	không
Địa hình	2	không	134.000	74	640	987	168	2.375	không
Địa hình	3	không	145.000	72	132	240	102	0.467	không

Bảng 4.9: Kết quả khảo sát biến thể A có trọng số*



Hình 4.10: Đánh đổi giữa số đỉnh đã duyệt và chi phí đường đi theo trọng số w

Từ thí nghiệm này có thể rút ra hai kết luận có giá trị thực tiễn. Thứ nhất, biến thể A* có trọng số chỉ thực sự tiết kiệm công việc khi tập CLOSED được khoá. Nếu vẫn cho phép mở lại đỉnh, việc tăng trọng số có thể phản tác dụng trên các lưới có trọng số địa hình. Thứ hai, việc tăng trọng số là một cách đánh đổi có kiểm soát: người thiết kế có thể chọn mức w phù hợp với ngân sách thời gian của ứng dụng, chấp nhận đường đi dài hơn một chút để có kết quả nhanh hơn nhiều. Trong ứng dụng mô phỏng đám đông hoặc trò chơi điện tử, mức đánh đổi này thường được chấp nhận để giữ khung hình ổn định.

4.9. Thí nghiệm 5: ảnh hưởng của cấu hình bốn hướng và tám hướng

Thí nghiệm thứ năm giữ nguyên thuật toán A* và bộ dữ liệu, chỉ thay đổi cấu hình di chuyển giữa bốn hướng và tám hướng. Khi chuyển sang tám hướng, chi phí của bước chéo được đặt bằng căn bậc hai của hai và hàm ước lượng được đổi từ khoảng cách Manhattan sang khoảng cách octile để giữ tính khả dụng. Thí nghiệm được chạy trên ba bộ dữ liệu là vật cản ngẫu nhiên, địa hình trọng số và mê cung.

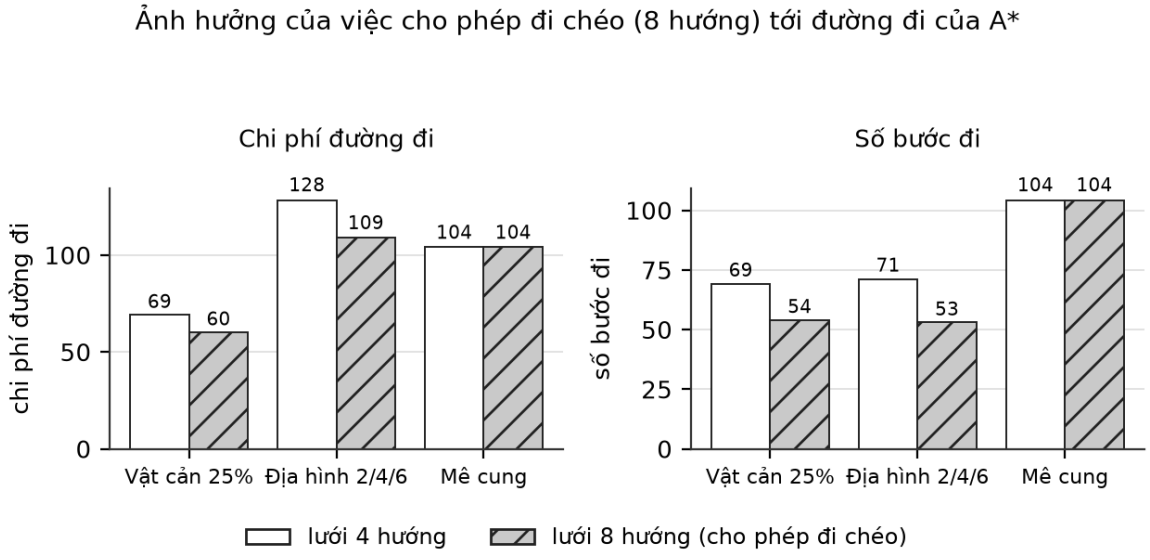
Kết quả trong Bảng 4.10 và Hình 4.11 cho thấy cấu hình tám hướng làm giảm cả chi phí đường đi lẫn số bước đi trên hai bộ dữ liệu có không gian mở. Trên bộ dữ liệu vật cản ngẫu nhiên, chi phí giảm từ sáu mươi chín xuống sáu mươi phẩy hai mươi mốt và số bước giảm từ sáu mươi chín xuống năm mươi bốn. Trên bộ dữ liệu địa hình trọng số, chi phí giảm từ một trăm hai mươi tám xuống một trăm linh tám phẩy tám bốn mươi mốt và số bước giảm từ bảy mươi mốt xuống năm mươi ba.

Trên bộ dữ liệu mê cung, hai cấu hình cho kết quả hoàn toàn giống nhau vì hành lang của mê cung chỉ cho phép đi theo bốn hướng.

Song song với việc giảm chi phí, cấu hình tám hướng làm tăng số đỉnh phải duyệt. Trên bộ dữ liệu vật cản ngẫu nhiên, số đỉnh duyệt tăng từ một trăm hai mươi bảy lên ba trăm lẻ năm, tức là tăng hơn hai lần. Nguyên nhân là hệ số phân nhánh tăng từ bốn lên tám, do đó với mỗi đỉnh được duyệt, thuật toán phải sinh và kiểm tra gấp đôi số đỉnh kề. Kết quả này cho thấy việc cho phép đi chéo là một quyết định thiết kế có hai mặt, cần cân nhắc theo yêu cầu cụ thể của ứng dụng.

Bộ dữ liệu	**Cấu hình**	**h(n)**	**Chi phí**	**Ô**	**Bước**	**Duyệt**	**Sinh**	**TG (ms)**	**Tối ưu**
Vật cản 25%	4 hướng	manh.	69.000	70	69	127	189	0.557	có
Vật cản 25%	8 hướng	octile	60.213	55	54	305	430	1.520	có
Địa hình	4 hướng	manh.	128.000	72	71	1184	1483	5.103	có
Địa hình	8 hướng	octile	108.841	54	53	1172	1668	9.020	có
Mê cung	4 hướng	manh.	104.000	105	104	111	112	0.290	có
Mê cung	8 hướng	octile	104.000	105	104	111	112	0.391	có

Bảng 4.10: Kết quả so sánh cấu hình bốn hướng và tám hướng



Hình 4.11: Ảnh hưởng của việc cho phép đi chéo tới chi phí và số bước đi

4.10. Phân tích tổng hợp và độ tin cậy của số liệu

Kết quả của năm nhóm thí nghiệm cho phép rút ra bốn nhận định tổng hợp. Nhận định thứ nhất là A* đạt được mức giảm số đỉnh duyệt rất lớn so với tìm kiếm theo chi phí đều trên các bản đồ không phải hành lang đơn tuyến, trong khi chất lượng đường đi hoàn toàn không suy giảm. Nhận định thứ hai là lợi ích đó phụ thuộc vào cấu trúc của bản đồ, và trở nên bằng không trên các bản đồ chỉ có một lối đi duy nhất. Nhận định thứ ba là hàm ước lượng cần được chọn phù hợp với cấu hình di chuyển, nếu không sẽ mất tính tối ưu hoặc mất phần lớn lợi ích về số đỉnh duyệt.

Nhận định thứ tư liên quan tới mức giảm trung bình. Tính trên sáu bộ dữ liệu, số đỉnh mà A* phải duyệt trung bình bằng khoảng 54 phần trăm số đỉnh của tìm kiếm theo chi phí đều. Chỉ số này bị chi phối mạnh bởi bộ dữ liệu hành lang xoắn ốc, nơi A* không thể giảm được đỉnh nào vì chỉ tồn tại một lối đi. Nếu loại bộ dữ liệu đặc biệt này ra khỏi phép tính, tỉ lệ trên giảm xuống còn khoảng 45 phần trăm. Việc báo cáo cả hai con số giúp tránh kết luận sai lệch do một bộ dữ liệu bất thường gây ra.

Về độ tin cậy của số liệu, có ba điểm cần được nêu rõ để người đọc đánh giá đúng các bảng và biểu đồ trong chương này. Thứ nhất, số đo thời gian trong các bảng thí nghiệm là thời gian bộ máy tìm kiếm chạy, không bao gồm thời gian vẽ lên màn hình. Số hiển thị trên giao diện ở Hình 4.3 và Hình 4.4 cũng được tính theo nguyên tắc tương tự, chỉ cộng thời gian thực thi của bộ máy tìm kiếm và loại bỏ thời gian giao diện chờ giữa các bước, do đó không bị phóng đại khi mô phỏng chậm.

Thứ hai, các phép đo với mỗi cấu hình chỉ được thực hiện một lần cho mỗi bộ dữ liệu. Vì mọi bộ dữ liệu đều được sinh xác định theo hạt giống ngẫu nhiên cố định và thuật toán không dùng yếu tố ngẫu nhiên trong khi duyệt, kết quả lặp lại được tuyệt đối về số đỉnh duyệt, số đỉnh sinh và đường đi. Chỉ số thời gian có dao động giữa các lần chạy do ảnh hưởng của hệ điều hành, nhưng mức dao động nhỏ so với sự khác biệt giữa các thuật toán. Thứ ba, chi phí đường đi được tính bằng hàm tính chi phí trên chính đường đi trả về, không lấy từ giá trị ưu tiên của hàng đợi, do đó loại trừ được khả năng báo nhầm chi phí khi thuật toán bỏ qua trọng số địa hình như trường hợp tìm kiếm theo chiều rộng.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đã đạt được

Đối chiếu với mục tiêu đặt ra ở mục 1.5, đề tài đã hoàn thành cả bốn nhóm công việc. Về mặt lý thuyết, báo cáo đã trình bày đầy đủ cơ sở của thuật toán A* gồm mô hình đồ thị của lưới, định nghĩa hàm ưu tiên, điều kiện khả dụng và nhất quán của hàm ước lượng, chứng minh tính tối ưu, phân tích độ phức tạp thời gian và bộ nhớ, cùng nhận xét về phạm vi hiệu quả của thuật toán. Các nội dung này tạo nền tảng để giải thích mọi kết quả đo đạc ở Chương 4.

Về mặt cài đặt, đề tài đã xây dựng được một chương trình hoàn chỉnh gồm thư viện tìm kiếm độc lập, giao diện đồ hoạ cho phép chỉnh sửa bản đồ và quan sát quá trình tìm kiếm theo từng bước, kịch bản thực nghiệm tự động và bộ kiểm thử gồm hai mươi tám ca. Chương trình chạy được ở cả hai chế độ dòng lệnh và giao diện, chỉ dùng thư viện chuẩn của Python nên không cần bước cài đặt thêm gói phụ thuộc.

Về mặt dữ liệu và thực nghiệm, đề tài đã xây dựng sáu bộ dữ liệu mẫu bao phủ các cấu trúc bản đồ khác nhau và thực hiện năm nhóm thí nghiệm đo đạc có định lượng. Kết quả thực nghiệm đã kiểm chứng được ba kết luận lý thuyết quan trọng gồm tính tối ưu của A* khi hàm ước lượng khả dụng, ảnh hưởng của độ chặt hàm ước lượng tới số đỉnh phải duyệt, và sự đánh đổi giữa tốc độ và chất lượng đường đi trong biến thể A* có trọng số.

Về mặt minh chứng, toàn bộ số liệu dùng để viết báo cáo đều do chương trình sinh ra và được lưu lại dưới dạng tệp CSV, JSON, Markdown và HTML trong thư mục kết quả của đề tài. Nhờ vậy, mọi bảng và biểu đồ trong báo cáo đều có thể kiểm chứng lại bằng cách chạy lại kịch bản thực nghiệm, đây là điểm được đặc biệt chú trọng để đảm bảo tính trung thực của báo cáo.

5.2. Hạn chế của đề tài

Hạn chế thứ nhất là phạm vi không gian tìm kiếm còn nhỏ. Các bộ dữ liệu mẫu có kích thước tối đa sáu mươi nhân ba mươi ô, do đó các vấn đề về bộ nhớ của A* chưa bộc lộ. Trên các bài toán thực tế với bản đồ hàng triệu ô, chi phí bộ nhớ của tập OPEN và tập CLOSED sẽ là yếu tố giới hạn chính, và báo cáo chưa có số liệu thực nghiệm cho tình huống này.

Hạn chế thứ hai là đề tài chưa xét bản đồ thay đổi theo thời gian. Trong các ứng dụng như robot tự hành, bản đồ có thể được cập nhật trong lúc di chuyển và đường đi phải được tính lại cục bộ. Việc cài đặt một thuật toán tìm kiếm gia tăng như D* Lite và so sánh với việc chạy lại A* từ đầu là một hướng nghiên cứu tự nhiên nhưng nằm ngoài phạm vi đã đăng ký của đề tài.

Hạn chế thứ ba là các phép đo thời gian chỉ được thực hiện một lần cho mỗi cấu hình nên chưa có phân tích thống kê về phương sai. Mặc dù kết quả về số đỉnh duyệt và đường đi lặp lại được tuyệt đối, các kết luận về thời gian chỉ nên được hiểu là so sánh tương đối giữa các thuật toán chứ không phải là thông số tuyệt đối cho một cấu hình phần cứng cụ thể. Hạn chế thứ tư là giao diện chỉ chạy trên nền tảng có Tkinter, và việc mô phỏng trên lưới quá lớn sẽ làm tốc độ vẽ chậm đi đáng kể dù bộ máy tìm kiếm vẫn nhanh.

5.3. Hướng phát triển

Hướng phát triển thứ nhất là cài đặt thêm các biến thể tối ưu chuyên biệt cho lưới như tìm kiếm theo bước nhảy Jump Point Search và A* phân cấp, rồi so sánh trực tiếp với A* trên cùng bộ dữ liệu. Đây là hướng phát triển có giá trị học thuật cao vì cho phép định lượng thêm lợi ích của việc khai thác cấu trúc đều đặn của lưới. Hướng thứ hai là bổ sung tìm kiếm gia tăng cho bản đồ động, trong đó người dùng có thể thêm hoặc bỏ tường và quan sát chương trình cập nhật lại đường đi một cách cục bộ.

Hướng phát triển thứ ba là mở rộng không gian tìm kiếm lên ba chiều dạng khối lập phương rời rạc, áp dụng cho bài toán lập kế hoạch bay của thiết bị bay không người lái hoặc di chuyển trong nhà nhiều tầng. Hướng thứ tư là tăng cường khả năng trực quan hoá bằng cách hiển thị bản đồ nhiệt của giá trị f và số lần mở lại đỉnh, giúp người học thấy được ảnh hưởng của việc chọn hàm ước lượng tới từng ô trên lưới.

Hướng phát triển thứ năm là bổ sung chế độ so sánh nhiều thuật toán trên cùng một khung hình với thanh tiến trình đồng bộ, và chế độ xuất video mô phỏng để phục vụ thuyết trình. Hướng thứ sáu là đóng gói chương trình thành ứng dụng chạy độc lập để người dùng không cần cài Python, cùng với tài liệu hướng dẫn sử dụng ngắn gọn đi kèm.

TÀI LIỆU THAM KHẢO

[1] Hart, P. E., Nilsson, N. J., Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107.

[2] Russell, S., Norvig, P. (2021). Artificial Intelligence: A Modern Approach, Fourth Edition. Pearson.

[3] Dechter, R., Pearl, J. (1985). Generalized Best-First Search Strategies and the Optimality of A*. Journal of the ACM, 32(3), 505-536.

[4] Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. Numerische Mathematik, 1, 269-271.

[5] Hart, P. E., Nilsson, N. J., Raphael, B. (1972). Correction to A Formal Basis for the Heuristic Determination of Minimum Cost Paths. SIGART Newsletter, 37, 28-29.

[6] Patel, A. (n.d.). Red Blob Games: Introduction to A* Pathfinding. Tài liệu trực tuyến, truy cập tháng 10 năm 2026.

[7] Koenig, S., Likhachev, M. (2002). D* Lite. Proceedings of the AAAI Conference on Artificial Intelligence, 476-483.

[8] Harabor, D., Grastien, A. (2011). Online Graph Pruning for Pathfinding on Grid Maps. Proceedings of the AAAI Conference on Artificial Intelligence, 1114-1119.

[9] Python Software Foundation (2026). The Python Standard Library: heapq, math, deque, unittest. Tài liệu trực tuyến, truy cập tháng 10 năm 2026.

[10] Python Software Foundation (2026). Tkinter: Python interface to Tcl/Tk. Tài liệu trực tuyến, truy cập tháng 10 năm 2026.

[11] Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2022). Introduction to Algorithms, Fourth Edition. MIT Press.

[12] Cài đặt mẫu A* trên Wikipedia tiếng Anh, mục A* search algorithm, phiên bản tháng 9 năm 2026. Tài liệu trực tuyến, dùng để đối chiếu mã giả.

PHỤ LỤC A: MÃ NGUỒN CỐT LÕI

Phụ lục này trích hai phần mã nguồn quan trọng nhất của chương trình: hàm tìm kiếm của bộ máy A* và phần cập nhật trạng thái tìm kiếm của giao diện. Mã nguồn được trích nguyên văn từ tệp trong thư mục dự án, chỉ lược bỏ các phần xử lý giao diện không liên quan trực tiếp tới thuật toán.

A.1. Hàm tìm kiếm của bộ máy A*, UCS, Greedy và BFS

```
# Trích astar/search.py: bộ máy A*, UCS, Greedy (rút gọn phần khai báo)
def search_steps(grid, start, goal, mode=MODE_ASTAR,
                 heuristic=H.DEFAULT_NAME, weight=1.0,
                 tie_break=True, keep_sets=False, allow_reopen=True):
    grid: Grid,
    start: Node,
    goal: Node,
    mode: str = MODE_ASTAR,
    heuristic: str = H.DEFAULT_NAME,
    weight: float = 1.0,
    tie_break: bool = True,
    keep_sets: bool = False,
    allow_reopen: bool = True,
) -> Iterator[tuple]:
    """Chạy tìm kiếm và phát ra sự kiện.

    allow_reopen:
        True  - cho phép "mở lại" đỉnh đã đóng khi tìm được g(n) nhỏ hơn
                (cần thiết khi h(n) không nhất quán; trung thực về mặt lý thuyết).
        False - khoá tập CLOSED. Chạy nhanh hơn, nhưng với h(n) không khả dụng
                (ví dụ weighted A* với w > 1) có thể cho đường DƯỚI tối ưu.

    Sự kiện:

    Sự kiện:
        ("expand",  node, g)            - đỉnh vừa được lấy ra khỏi OPEN
        ("frontier", node, g)           - đỉnh mới/cập nhật trong OPEN
        ("reuse",   node, g)            - đỉnh đã đóng được mở lại
        ("done",    SearchResult)       - kết thúc (luôn là sự kiện cuối)
    """
    if heuristic == "dijkstra":
        heuristic = "zero"
    h_fn = H.get(heuristic)

    result = SearchResult(
        algorithm=mode, heuristic=heuristic if mode != MODE_UCS else "-",
        weight=float(weight), diagonal=grid.diagonal,
allow_reopen=bool(allow_reopen),
    )
    t0 = time.perf_counter()

    # --- Kiểm tra đầu vào ------------------------------------------------
    if grid.is_blocked(start) or grid.is_blocked(goal):
        result.time_ms = (time.perf_counter() - t0) * 1000.0
        if keep_sets:
            result.closed, result.open_left = set(), set()
```

```
    yield ("done", result)
    return

if start == goal:
    result.found, result.path, result.cost = True, (start,), 0.0
    result.time_ms = (time.perf_counter() - t0) * 1000.0
    yield ("expand", start, 0.0)
    if keep_sets:
        result.closed, result.open_left = {start}, set()
    yield ("done", result)
    return

# --- BFS: hàng đợi FIFO, mọi bước tính bằng 1 (bỏ qua trọng số) -------
if mode == MODE_BFS:
    yield from _bfs_steps(grid, start, goal, result, t0, keep_sets)
    return

# --- Các thuật toán có chi phí ----------------------------------------
gscore: dict[Node, float] = {start: 0.0}
parent: dict[Node, Node | None] = {start: None}
depth: dict[Node, int] = {start: 0}
closed: set[Node] = set()
heap: list[tuple[float, float, int, Node]] = []
counter = itertools.count()
scale = grid.min_step_cost

def priority(node: Node, g: float) -> tuple[float, float]:
    if mode == MODE_GREEDY:
        h = h_fn(node, goal) * scale
        return h, h
    h = h_fn(node, goal) * scale if h_fn is not H.zero else 0.0
    return g + weight * h, (h if tie_break else 0.0)

def push(node: Node, g: float) -> None:
    f, tie = priority(node, g)
    heapq.heappush(heap, (f, tie, next(counter), node))

push(start, 0.0)
result.generated = 1

while heap:
    f, _, _, node = heapq.heappop(heap)
    if node in closed:
        continue                        # bản ghi cũ, đã duyệt rồi

    closed.add(node)
    gnode = gscore[node]
    result.expanded += 1
    result.iterations += 1
    result.max_open = max(result.max_open, len(heap))
    result.max_depth = max(result.max_depth, depth[node])
    yield ("expand", node, gnode)

    if node == goal:
        path = _reconstruct(parent, goal)
        result.found = True
        result.path = path
        result.cost = _path_cost(grid, path)
        result.time_ms = (time.perf_counter() - t0) * 1000.0
        if keep_sets:
```

```
                result.closed = closed
                result.open_left = {n for _, _, _, n in heap if n not in closed}
            yield ("done", result)
            return
```

A.2. Phần cập nhật trạng thái mô phỏng của giao diện

```
    def _advance(self, budget: int) -> None:
        """Xử lý tối đa `budget` sự kiện từ generator (tô màu từng ô ngay lập
tức)."""
        if self.gen is None:
            return
        prev = self.current
        for _ in range(budget):
            t_call = time.perf_counter()
            try:
                event = next(self.gen)
            except StopIteration:  # pragma: no cover
                self.finished = True
                return
            # Chỉ cộng thời gian bộ máy tìm kiếm thực sự chạy (bỏ thời gian vẽ/nghỉ)
            self.gen_time_ms += (time.perf_counter() - t_call) * 1000.0
            kind = event[0]
            if kind == "expand":
                node, g = event[1], event[2]
                self.open_g.pop(node, None)
                self.closed[node] = g
                self.current = node
                if prev is not None and prev != node:
                    self._set_cell_color(
                        prev,
                        COLOR_CLOSED if prev in self.closed
                        else COLOR_OPEN if prev in self.open_g
                        else self._base_color(prev),
                    )
                prev = node
                self._set_cell_color(node, COLOR_CURRENT)
            elif kind == "frontier":
                node, g = event[1], event[2]
                self.open_g[node] = g
                if node not in self.closed:
                    self._set_cell_color(node, COLOR_OPEN)
            elif kind == "reuse":
                node, g = event[1], event[2]
                self.closed.pop(node, None)
                self.open_g[node] = g
                self._set_cell_color(node, COLOR_OPEN)
            elif kind == "done":
                self.result = event[1]
                self.path = tuple(self.result.path)
                self.finished = True
                return

    # ... (xem toàn bộ mã nguồn tại de48/gui.py)
```

A.3. Cách chạy chương trình

Chương trình gồm bốn điểm vào được gom trong tệp main.py. Lệnh không kèm tham số sẽ mở giao diện đồ hoạ. Lệnh với tham số demo chạy một ví dụ và in lưới cùng đường đi ra màn hình dòng lệnh. Lệnh với tham số exp chạy toàn bộ thực nghiệm và ghi kết quả vào thư mục kết quả. Lệnh với tham số test chạy bộ kiểm thử tự động và in kết quả từng ca.

```
python main.py          # mở giao diện mô phỏng
python main.py demo     # chạy ví dụ và in lưới ra console
python main.py exp      # chạy thực nghiệm, xuất results/
python main.py test     # chạy 28 ca kiểm thử tự động
python main.py info     # xem thông tin môi trường và dữ liệu mẫu
```